

Informations verarbeitung

Performanzsteigerungen und Speicher

Dr.-Ing. Tanja Harbaum

Institut für Technik der Informationsverarbeitung (ITIV)

Übersicht – Vorlesung Nr. 03

- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Speicherhierarchien
 - Überblick Technologien

Performanz-Steigerung – CISC

- RISC-V ist ein Beispiel für eine ISA mit einem **Reduced** Instruction Set.
- x86-64 ist ein Beispiel für eine ISA mit einem **Complex** Instruction Set
- x86 hat z.T. sehr aufwendige Instruktionen (ganze Blöcke aus dem Speicher kopieren, mehrere Register auf einmal setzen)

CISC	RISC
viele Instruktionen	wenige Instruktionen
aufwendige Instruktionen	einfache Instruktionen
variable Instruktionslänge	konstante Instruktionslänge
Arithmetik / Logik kann direkt auf Speicher angewendet werden	Arithmetik / Logik nur mit Registern ausführbar

[1] s.389

Diskussion RISC versus CISC

- Unterscheidung RISC/CISC ist für moderne CPUs zu „einfach“
 - Früher: x86 versus ARM
- Prozessknoten, Mikroarchitektur und ISA haben je einen großen Einfluss auf die Leistung eines Chips
 - Es kommt eher auf die Entwurfsentscheidung statt auf den Instruktionssatz an
- CISC- und RISC-CPU's haben sich seit Jahrzehnten einander angenähert
 - Moderne x86 Architekturen verfügen über "RISC-ähnlicher" Dekodierungsmethoden
 - ARM: Multicycle instructions

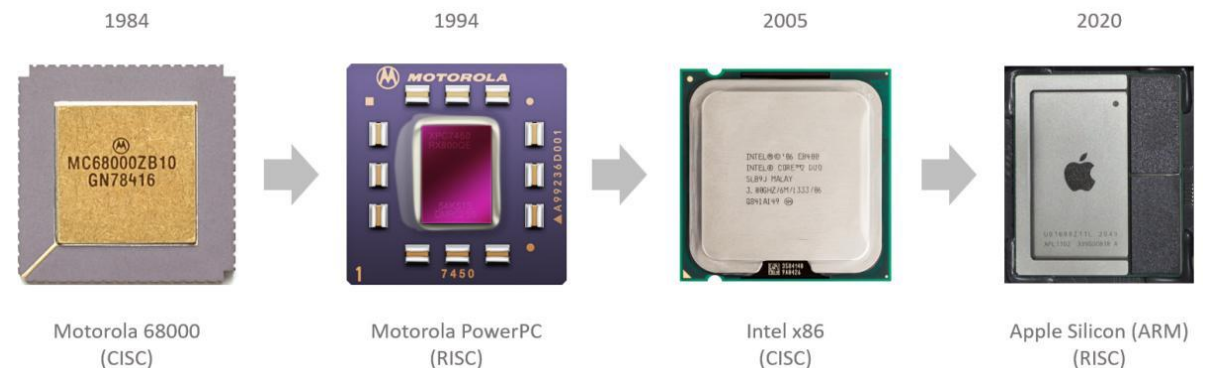
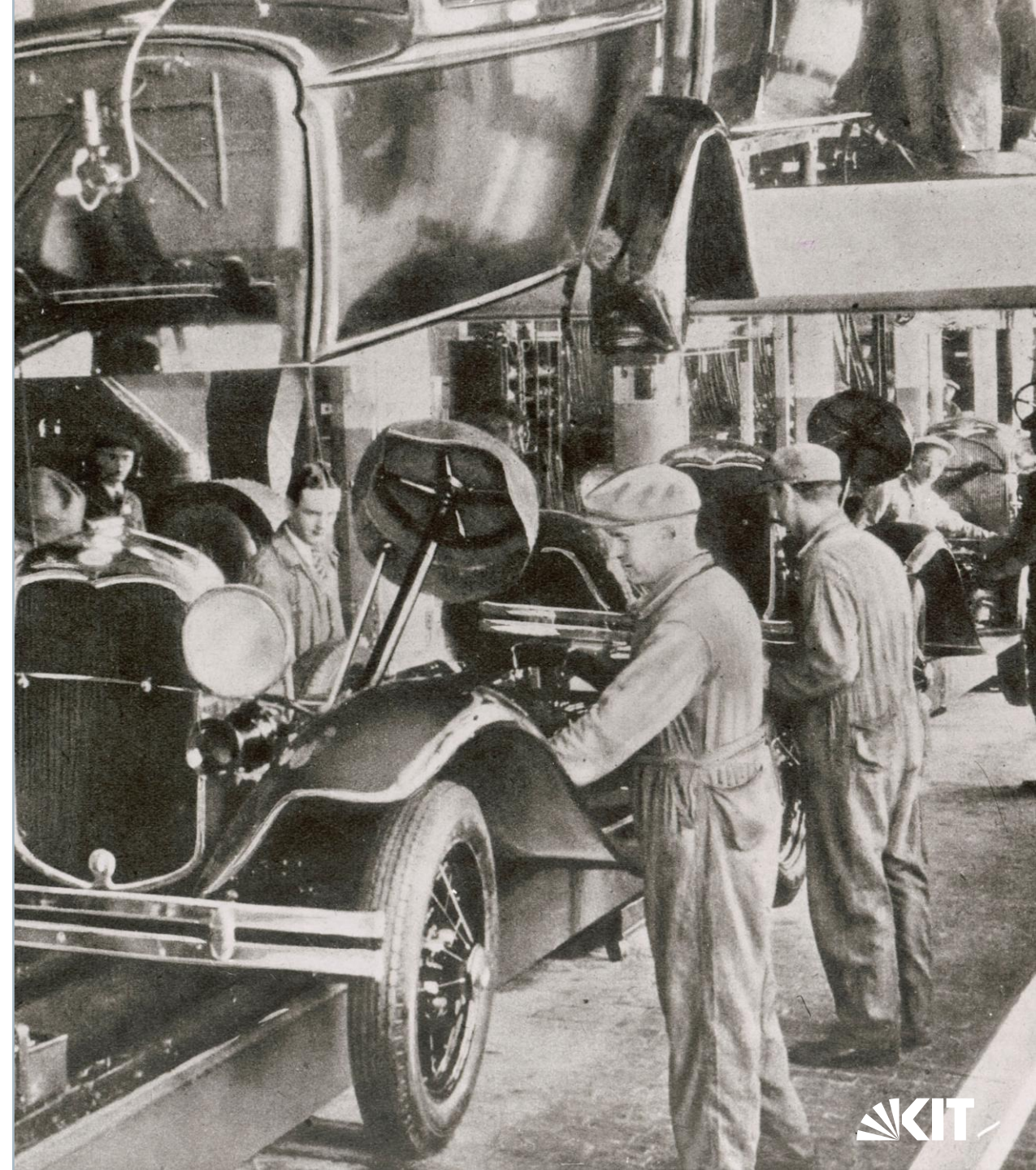
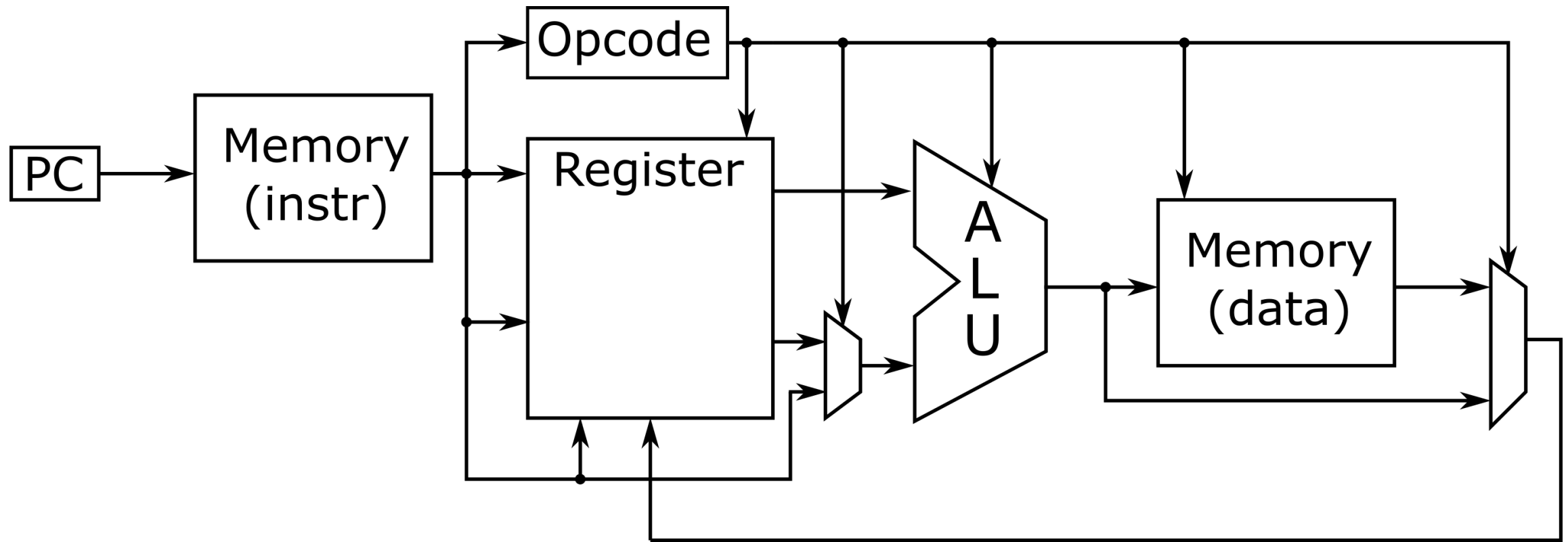


Image: <https://www.epcc.ed.ac.uk/whats-happening/articles/evolution-risc-architecture>

Pipelining - „Assembly line processing“

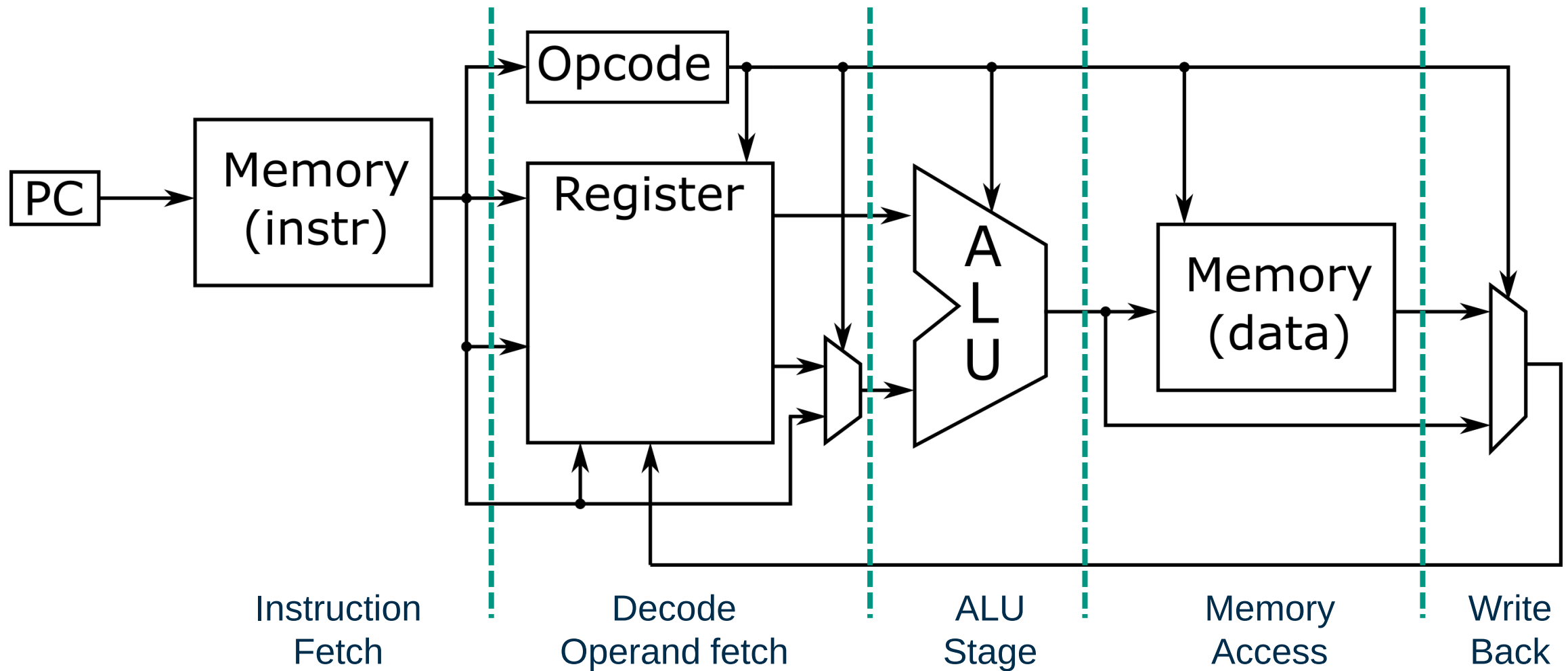
- „Pipelines accelerate the computer execution speed in the same way as Henry Ford revolutionized car manufacturing with the introduction of the assembly line.“
(Peter Wayner 1992)





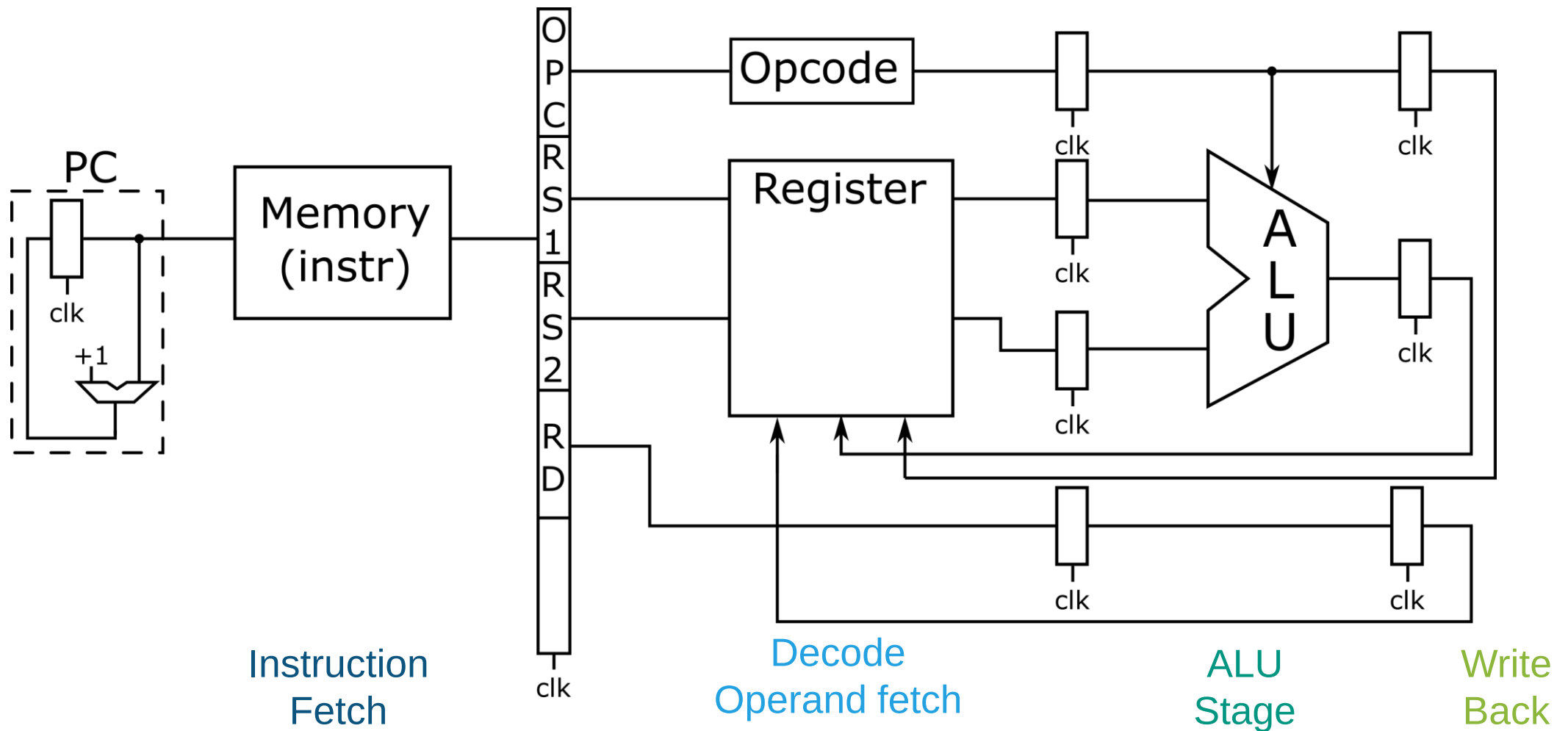
Ein einfacher Prozessor

- Ausführung einer Instruktion wird in mehrere Stages aufgeteilt

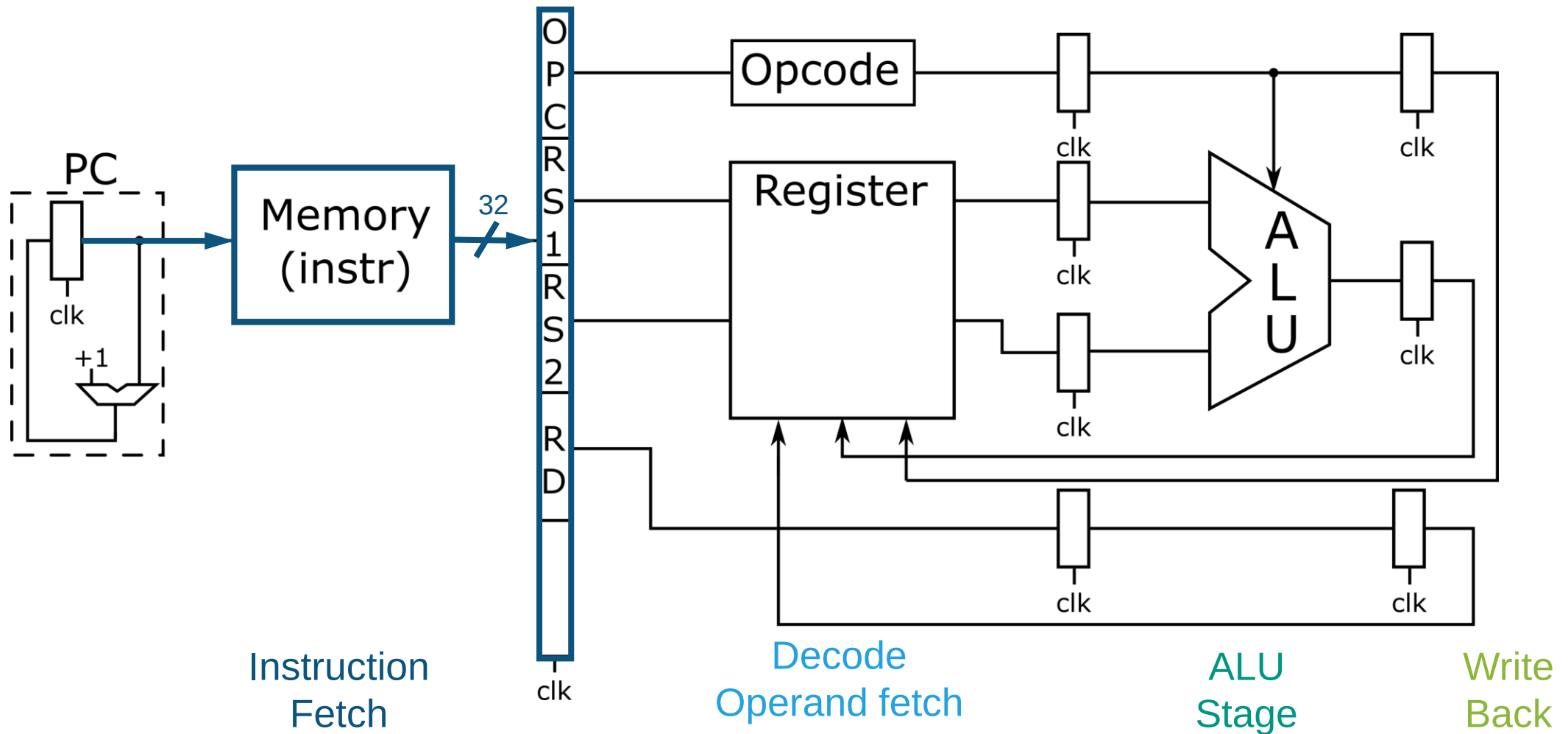


Ein einfacher Prozessor

- Ausführung einer Instruktion wird in mehrere Stages aufgeteilt

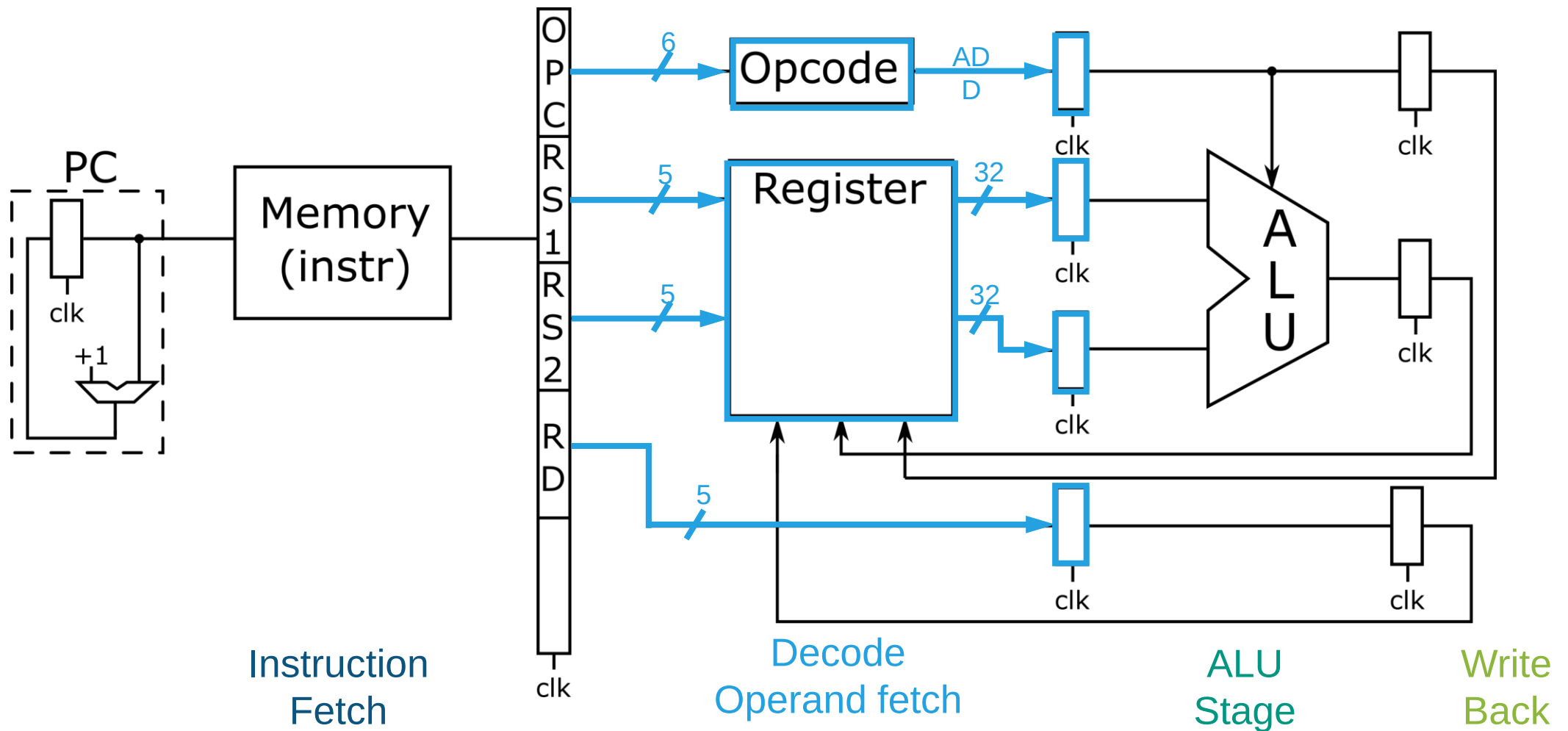


Pipelining: Beispiel ADD Instruktion



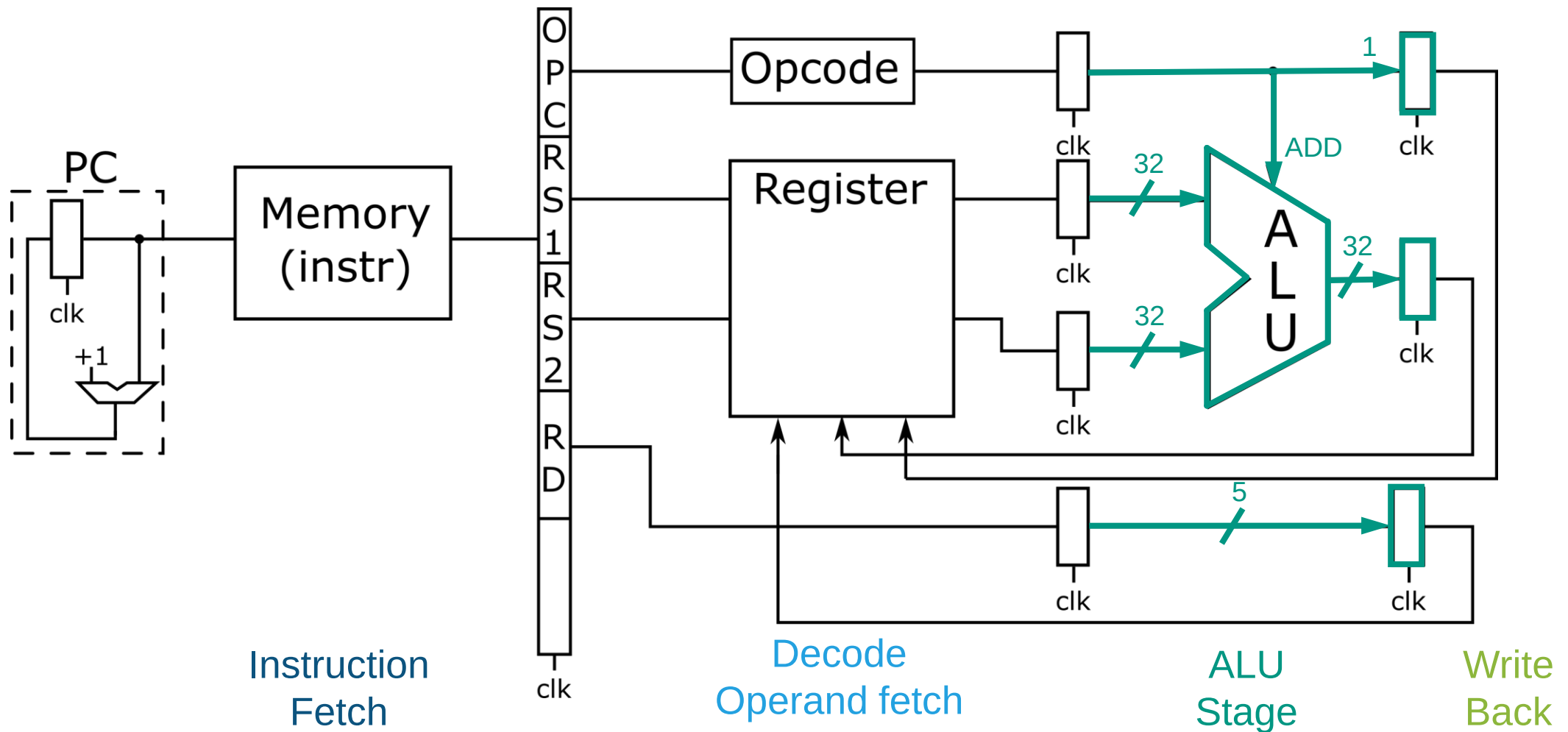
Pipelining: Beispiel ADD Instruktion

- 1. Schritt: Lade die Instruktion aus dem Speicher (FETCH)



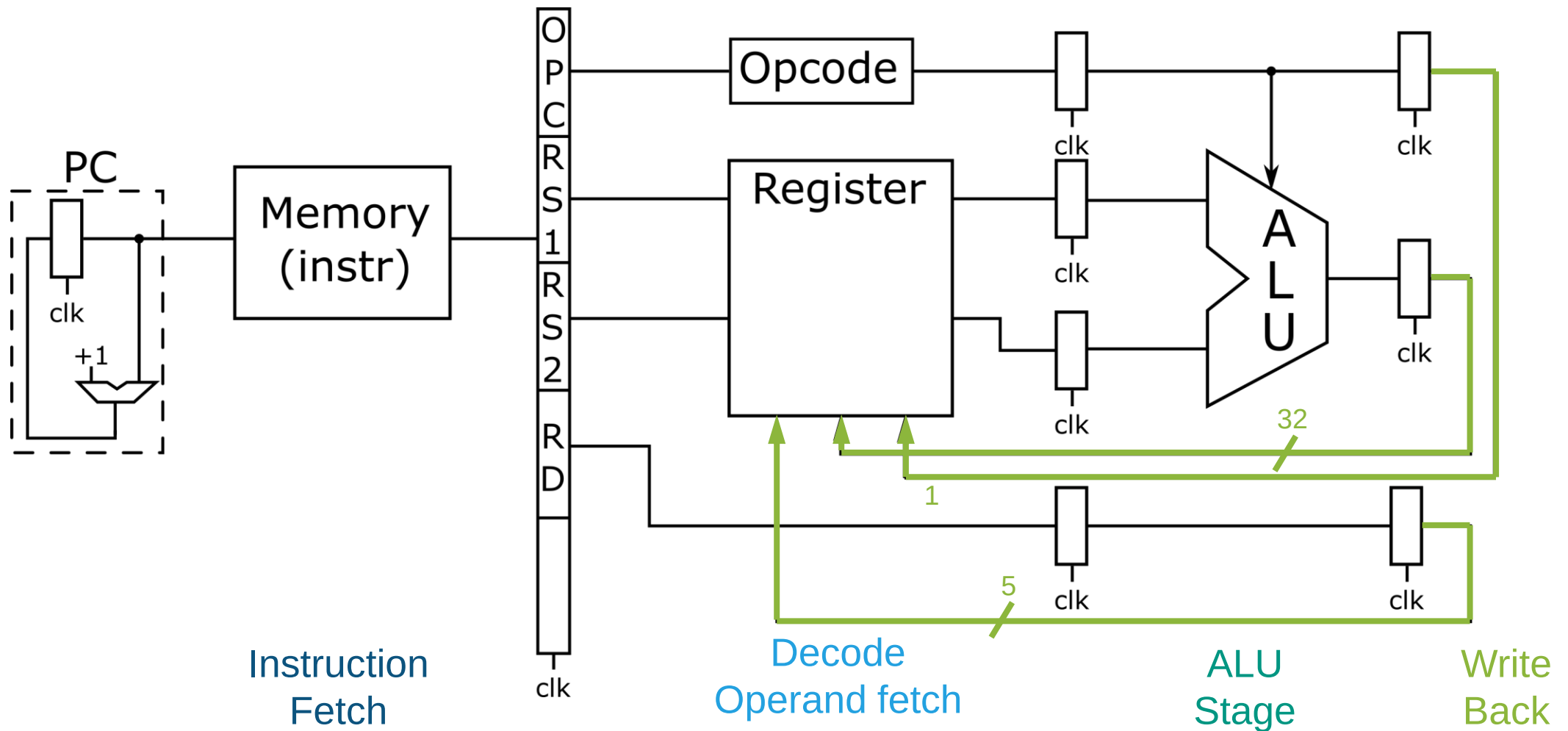
Pipelining: Beispiel ADD Instruktion

- 2. Schritt: Dekodiere Instruktion und lade Operanden (DECODE)



Pipelining: Beispiel ADD Instruktion

- 3. Schritt: Führe die Instruktion in ALU aus (EXECUTE)

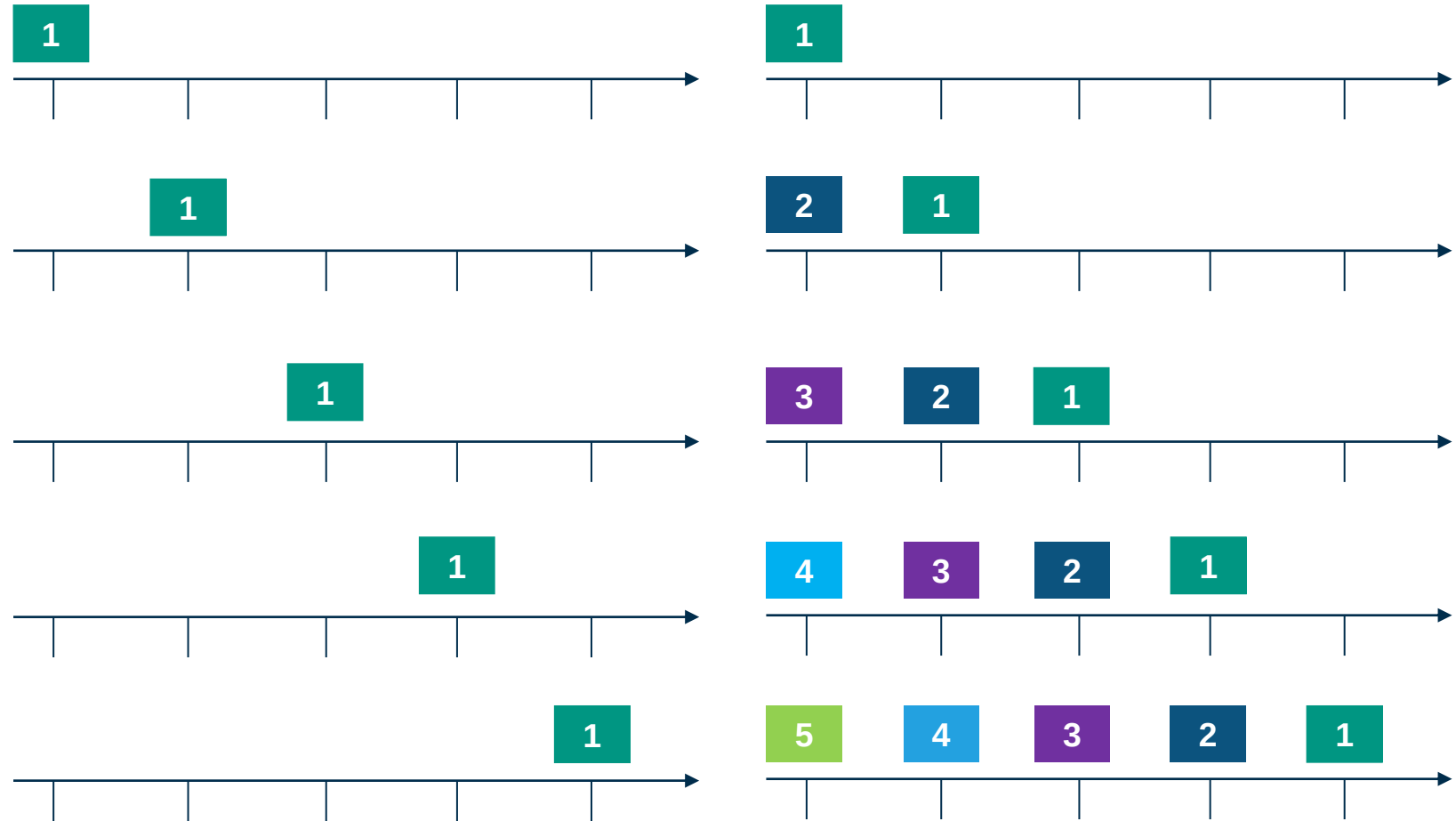


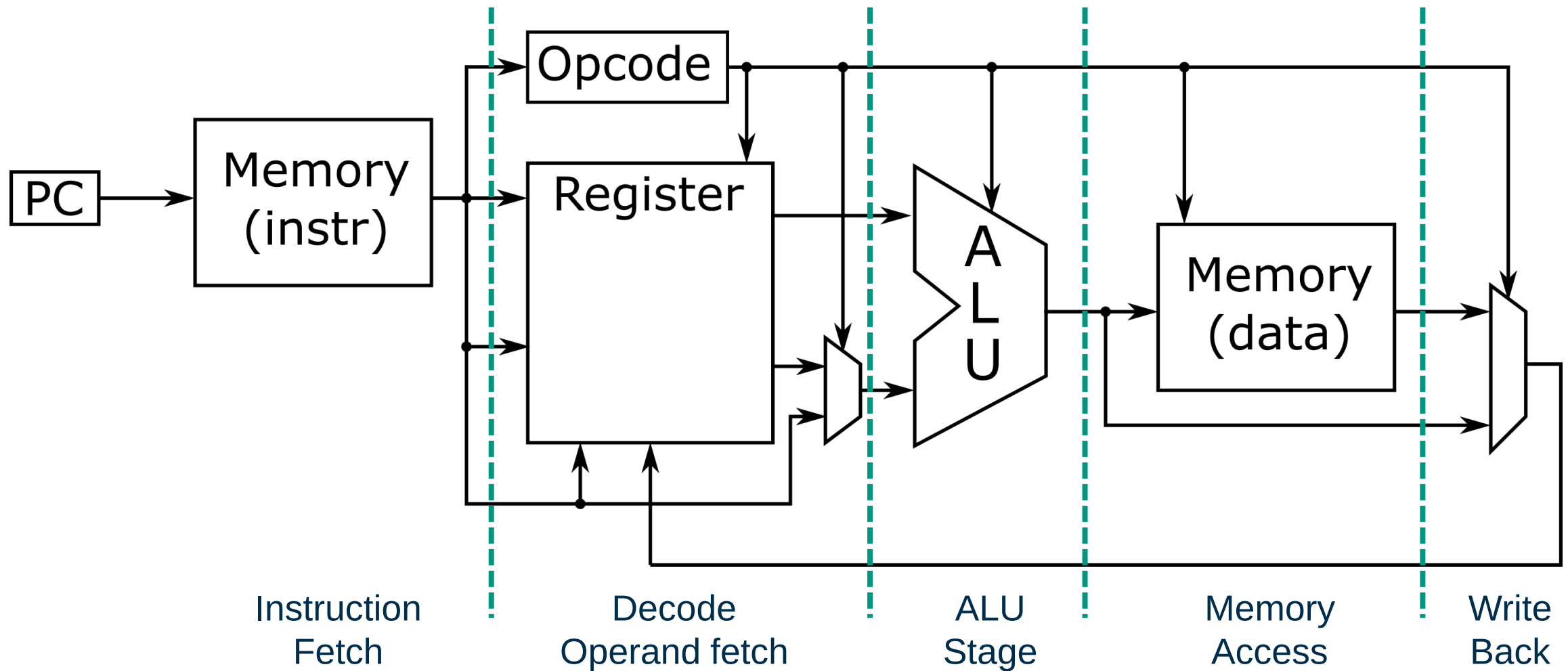
Pipelining: Beispiel ADD Instruktion

- 5. Schritt: Speichere Ergebnisse (WRITE BACK)

Durchsatz mit und ohne Pipeline

- Mehrere Instruktionen können gleichzeitig die Pipeline durchlaufen
 - Durchsatz steigt
 - Taktfrequenz steigt



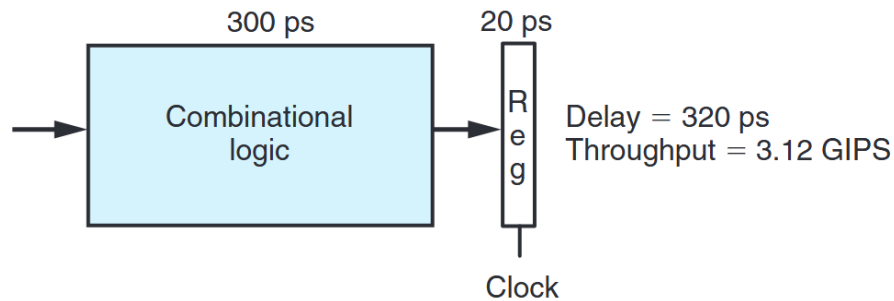


Ein einfacher Prozessor

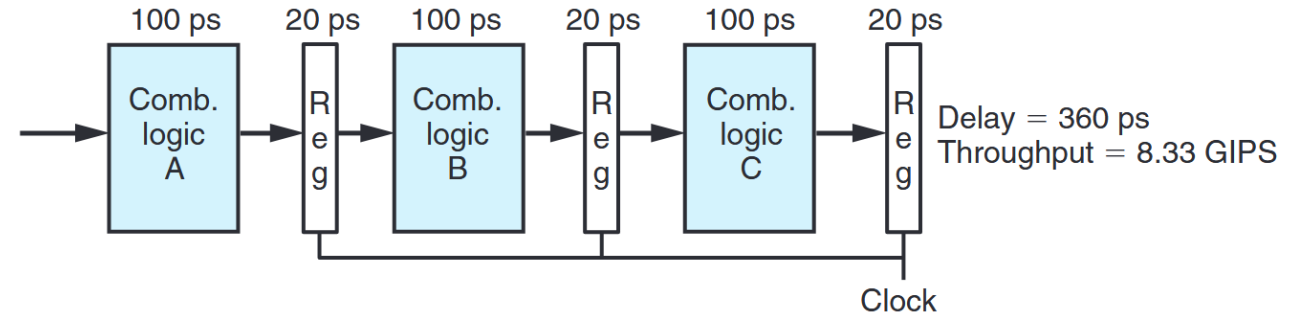
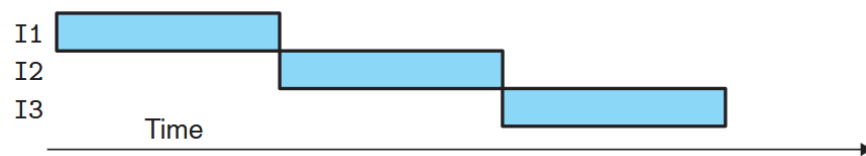
- Ausführung einer Instruktion wird in mehrere Stages aufgeteilt

Nachteil: Latenz

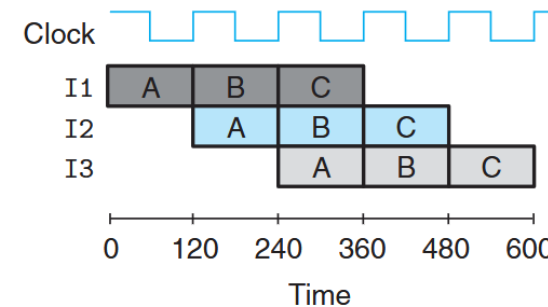
- Es werden zusätzliche Register zwischen den Stages benötigt
- Dadurch steigt die Latenz



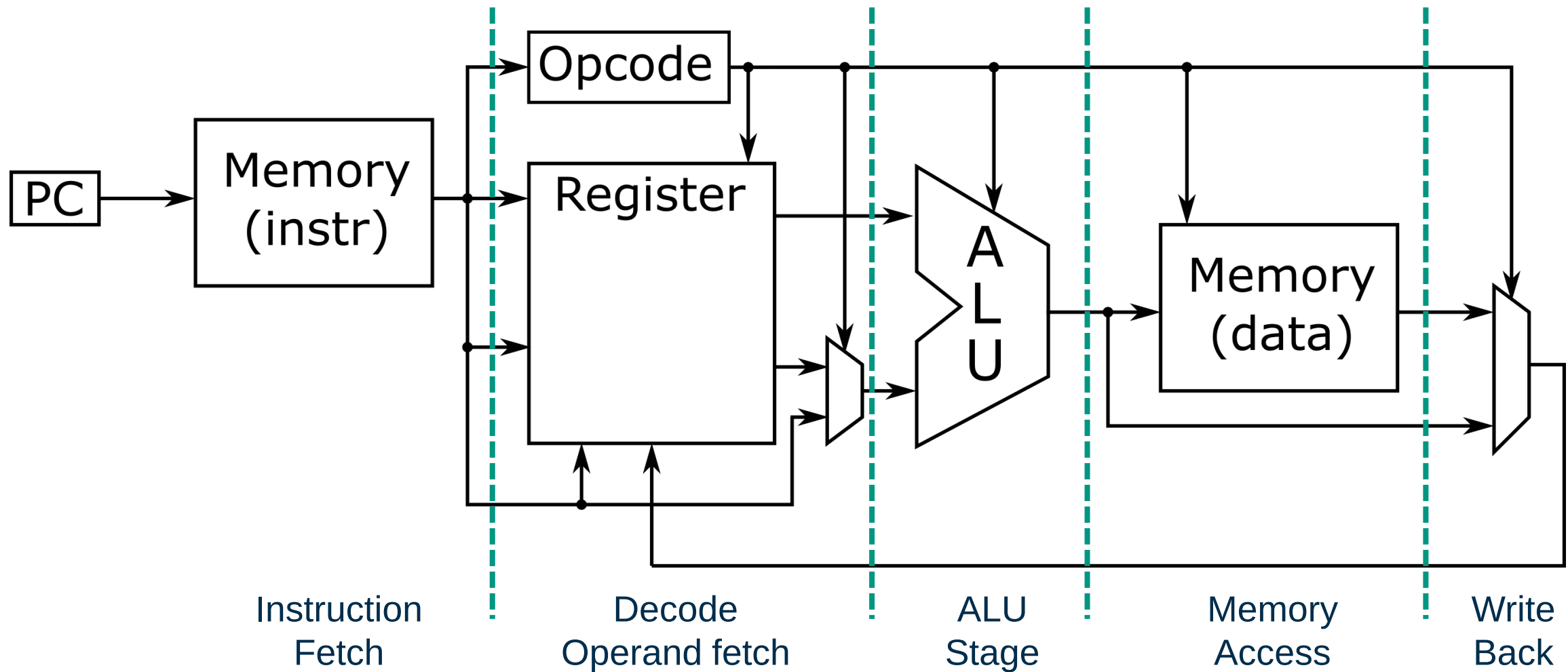
■ Durchsatz: $\frac{1 \text{ Instruktion}}{(300+20) \cdot 10^{-12} \text{ s}} = \frac{10^{12} \text{ Instruktionen}}{320 \text{ s}} = 3.12 \text{ GIPS}$



■ Durchsatz: $\frac{3 \text{ Instruktionen}}{3(100+20) \cdot 10^{-12} \text{ s}} = \frac{10^{12} \text{ Instruktionen}}{120 \text{ s}} = 8.33 \text{ GIPS}$



[1] S.442

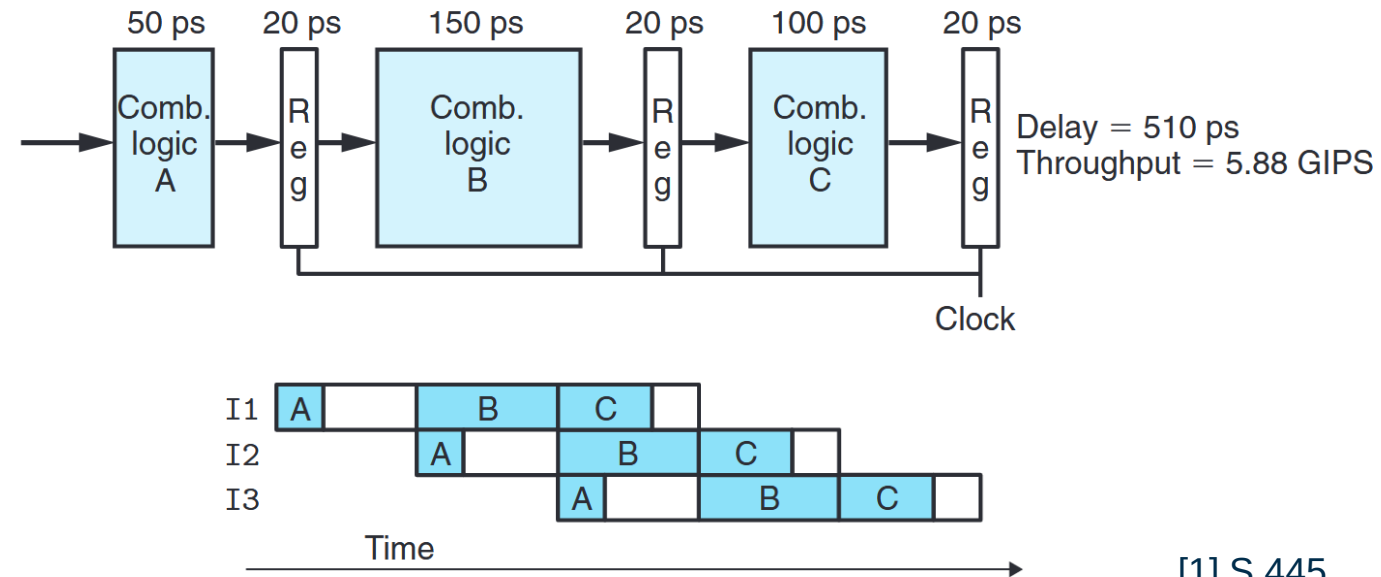
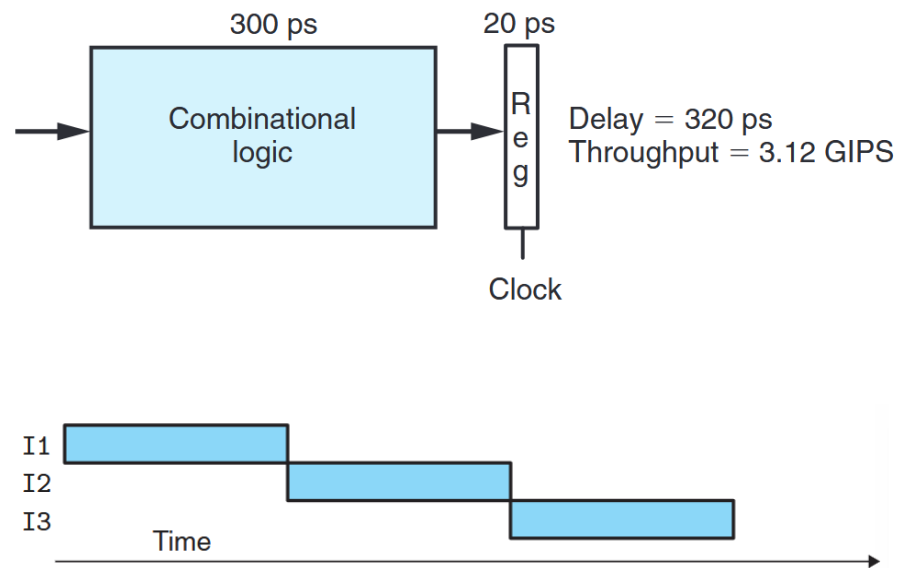


Ein einfacher Prozessor

- Ausführung einer Instruktion wird in mehrere Stages aufgeteilt

Nachteil: Stages müssen gleichförmig sein

- Man wartet immer auf die langsamste Stage
- Wenn die Stages nicht gleichförmig sind, müssen manche Stages warten
- Latenz steigt



[1] S.445

Durchsatz und Latenz einer Pipeline

- Latenz ist die Zeit, die ein Befehl benötigt, um alle k Pipeline-Stufen zu durchlaufen
- Der Durchsatz einer Pipeline wird definiert als die Anzahl der Befehle, die eine Pipeline pro Zeiteinheit verlassen können
 - Dieser Wert spiegelt die Rechenleistung einer Pipeline wieder
- Idealerweise wird ein Befehl in einer k -stufigen Pipeline in k Takten von k Stufen ausgeführt
- Wird in jedem Takt ein neuer Befehl geladen, dann werden zu jedem Zeitpunkt unter idealen Bedingungen k Befehle gleichzeitig behandelt und jeder Befehl benötigt k Takte, bis zum Verlassen der Pipeline

Pipeline Hazards

■ Strukturkonflikte

- Ergeben sich aus Ressourcenkonflikten
- Die Hardware kann nicht alle mögliche Kombinationen von Befehlen unterstützen, die sich in der Pipeline befinden können

■ Datenkonflikte

- Ergeben sich aus Datenabhängigkeiten zwischen Befehlen im Programm
- Instruktion benötigt das Ergebnis einer vorangehenden und noch nicht abgeschlossenen Instruktion in der Pipeline

■ Steuerkonflikte

- Treten bei Verzweigungsbefehlen und anderen Instruktionen auf, die den Befehlszähler verändern



Auflösung von Pipeline Konflikten

Softwarebasierend

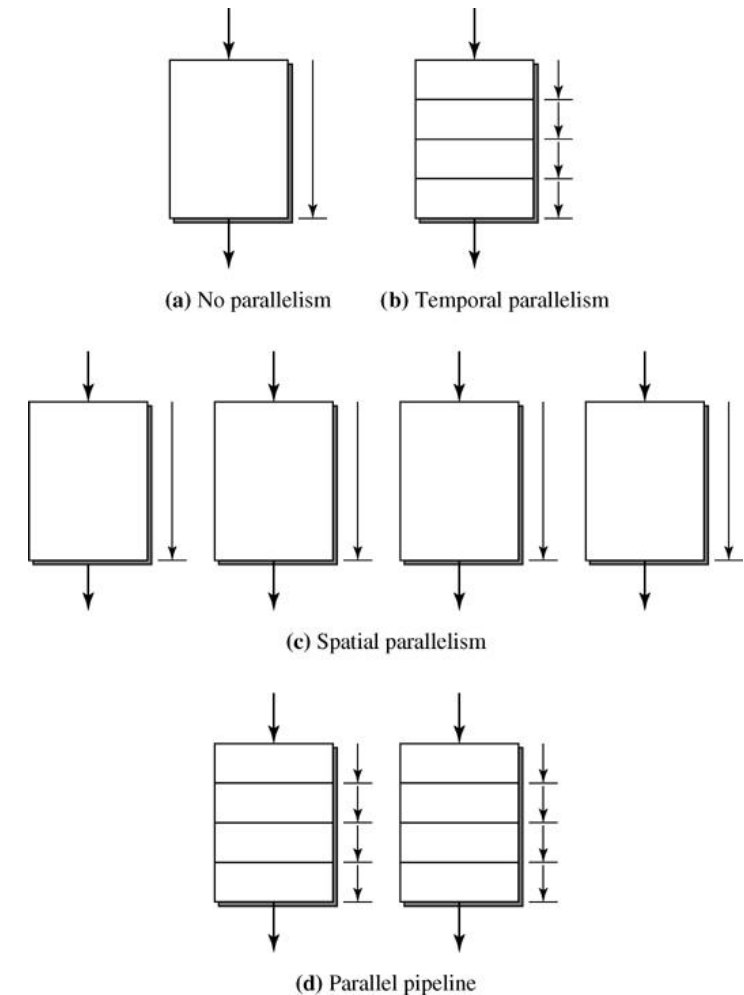
- Aufgabe des Compilers:
 - Erkennen von Datenkonflikten
 - Einfügen von Leeroperationen nach jedem Befehl, der einen Konflikt verursacht (NOP)
- Statische Verfahren:
 - Instruction Scheduling
 - Eliminieren von Leeroperationen
 - Umordnen der Befehle des Programms (Code-Optimierung)

Hardware-Lösungen (Dynamische Verfahren)

- Erkennen von Konflikten
 - Entsprechende Konflikterkennungslogik notwendig!
 - Techniken
 - Leerlauf der Pipeline (Stalling)
 - Forwarding

Parallelisierung

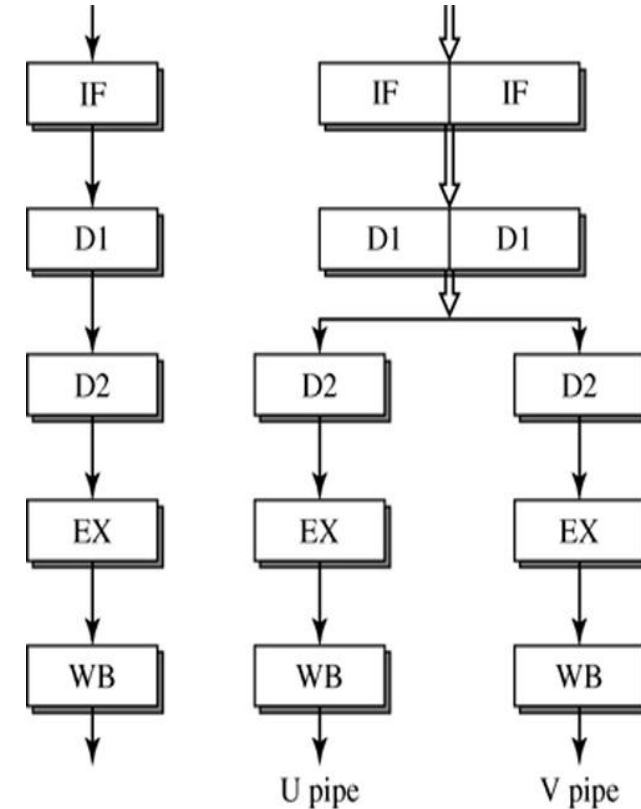
- (a) Keine Parallelität (Nonpipelined)
- (b) Zeitliche Parallelität (Pipelined)
- (c) Räumliche Parallelität (Mehrere Einheiten)
- (d) Zeitliche und Räumliche Parallelität
 - Parallele Pipeline



Source: Prof. Lizy Kurian John Univ. Austin, Texas

Superskalarität (Beispiel Pentium)

- Fetch
 - 16 Byte Befehlsdaten in den Prefetch Buffer laden
- Decode1
 - Befehlslänge und Befehlstyp ermitteln
- Decode2
 - Speicheradresse berechnen
 - Immediates generieren
- Execute
 - Register lesen
 - ALU Operation
 - Speicher lesen/schreiben
- Write-Back
 - Register aktualisieren



(a)
5-stufige i486
skalare Pipeline

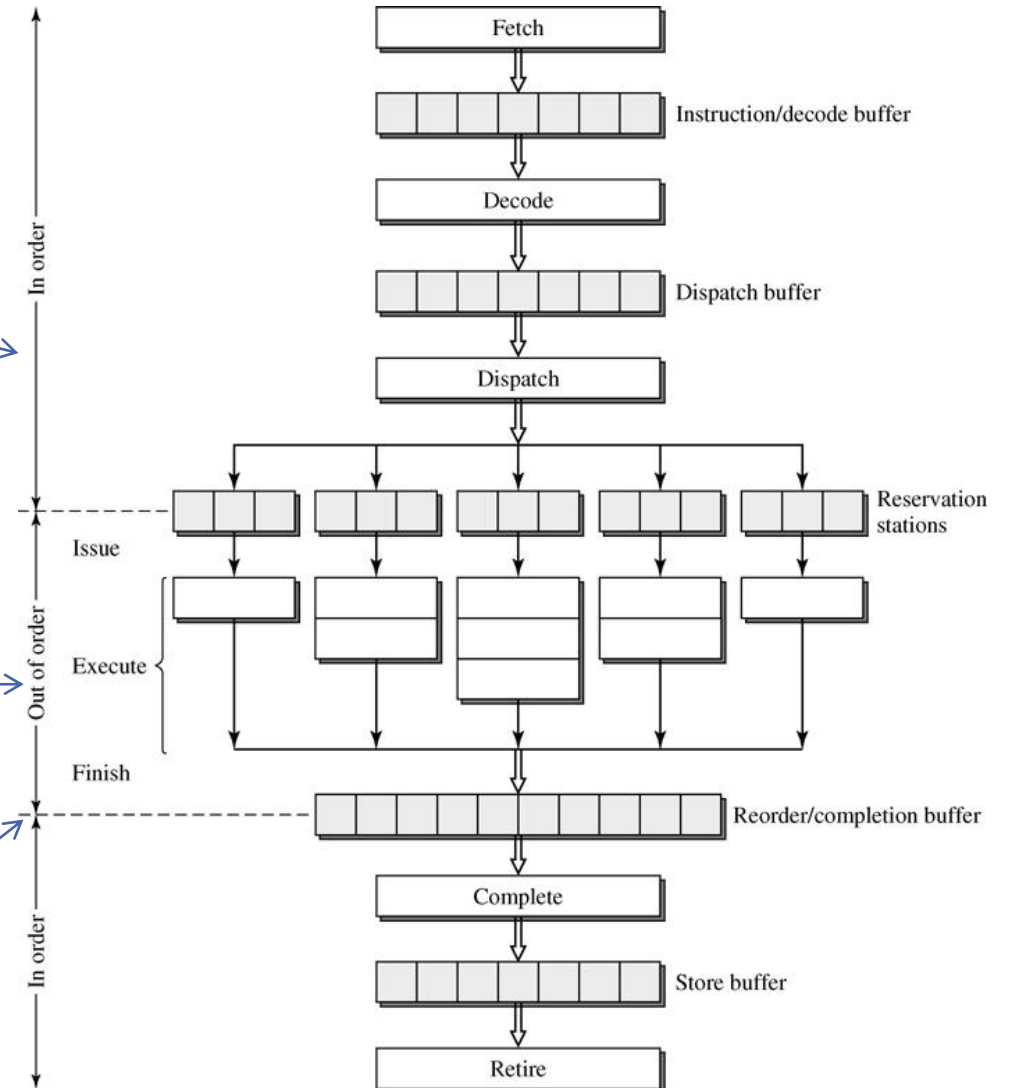
(b)
5-stufige Pentium
parallele Pipeline

Parallelisierbarkeit von Instruktionen

- Zwei Befehle können parallel ausgeführt werden, wenn:
 - Beide einfache Instruktionen sind
 - z.B. keine speziellen Einheiten einer bestimmten Pipeline benötigt werden
 - Instruktion 1 kein Sprung ist
 - Wenn das Ziel von Instruktion 1 nicht die Quelle von Instruktion 2 ist
 - Wenn das Ziel von Instruktion 1 nicht das Ziel von Instruktion 2 ist
- Wenn zwei Befehle nicht parallel ausgeführt werden können:
 - Instruktion 1 in erster Pipeline abarbeiten
 - Instruktion 2 im nächsten Zyklus abarbeiten
 - Wenn möglich, zusammen mit der darauf folgenden Instruktion

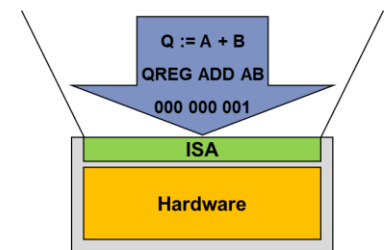
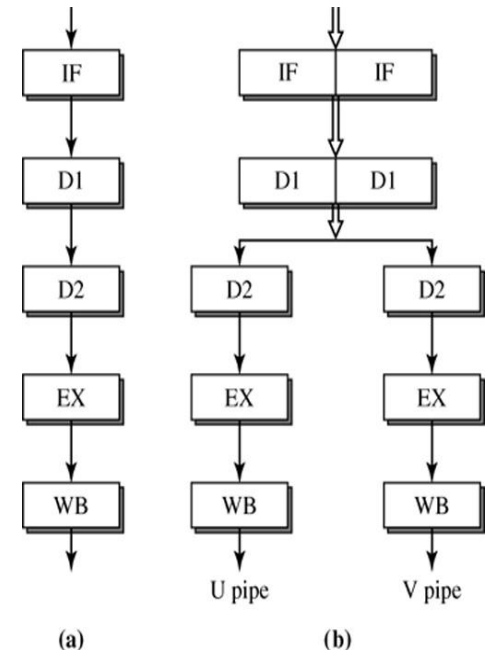
Dynamisches Scheduling – Prozessor Pipeline

- Prinzipieller Ablauf :
 - Zuweisen von Befehlen an Ausführungseinheiten
 - Reservation Stations
 - Weiterleiten der Befehle an Funktionseinheiten, sobald:
 - Operanden verfügbar sind
 - Funktionseinheit frei ist
 - Ablegen der Befehle in Programmreihenfolge in dem Reorder-Puffer



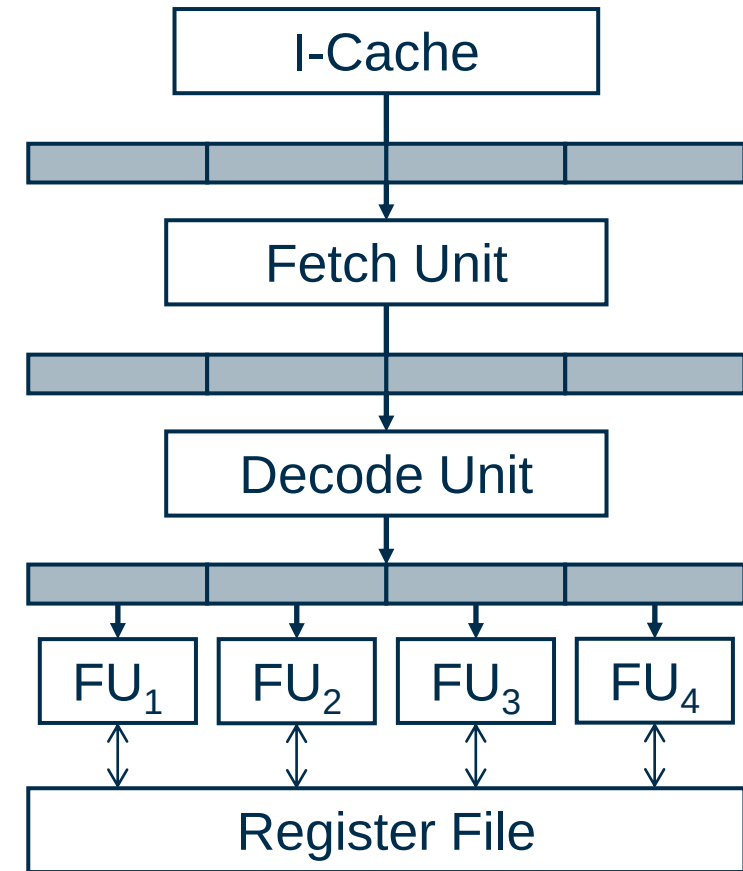
Präzisierung Begriff Superskalarität

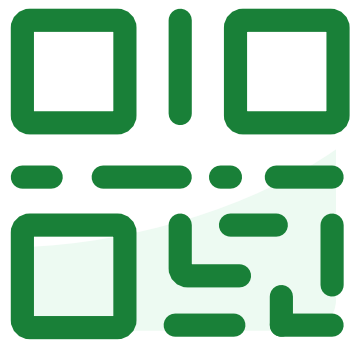
- Ein Skalarprozessor verarbeitet eine Information pro Taktzyklus (meist durch eine Pipeline-Architektur ermöglicht)
- Der Begriff „superskalar“ wird oft in unpräziser Weise benutzt, um Prozessoren mit mehreren parallelen Pipelines oder mehreren Ausführungseinheiten zu beschreiben
 - Gilt jedoch nur dann, wenn die Zuweisung der Instruktionen dynamisch erfolgt
 - Unterscheidung zwischen der Superskalar- und der VLIW-Technik
- (Super)skalartechnik ist eine Mikroarchitekturtechnik und hat keinen Einfluss auf die Befehlssatz-Architektur



Very Long Instruction Word (VLIW)

- Befehlssatz mit einer festen Anzahl von Befehlen
 - Parallelität auf Befehlsebene
 - Ein Maschinenbefehlswort fester Länge
 - Unabhängige Befehle
 - Jeder Befehl ist einer unabhängigen Funktionseinheit zugeordnet
 - Die Reihenfolge der VLIW-Befehlspakete ist fest vorgegeben
- Die Befehle in einem VLIW-Befehlswort werden gleichzeitig abgerufen, decodiert, zugewiesen und ausgeführt





**Join at slido.com
#33184904**



**Welche der folgenden Aussagen
über eine einzelne 5-stufige reale
Pipeline sind richtig?**



Was sind die Hauptgründe dafür, dass 50-stufige Pipelines in der Praxis nicht zum Einsatz kommen?



Welche der folgenden Aussagen beschreibt den Hauptunterschied zwischen einem skalaren und einem superskalaren Prozessor?



Was ist der größte Unterschied zwischen Superskalarität und VLIW?

Übersicht – Vorlesung Nr. 03

- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Speicherhierarchien
 - Überblick Technologien

Flynn'sche Taxonomie

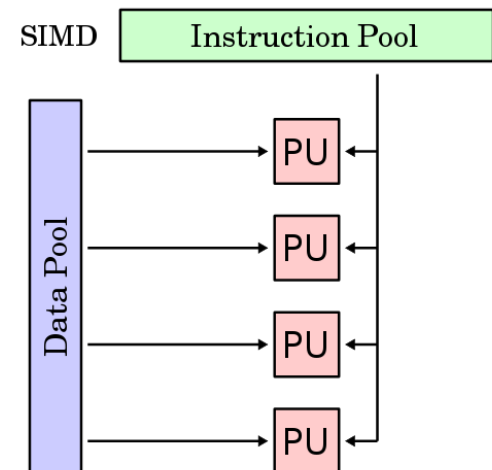
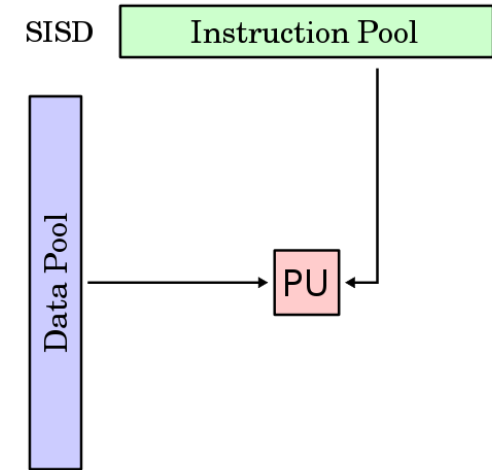
Flynn's Classification	Single Instruction	Multiple Instruction
Single Data	SISD	MISD
Multiple Data	SIMD	MIMD

- Wie viele Instruktionen werden gleichzeitig ausgeführt?
 - Anzahl der Instruktionsströme (Hauptspeicher ↔ Prozessor)
- Wie viele Datenworte verarbeitet eine Instruktion?
 - Anzahl der Datenströme (Hauptspeicher ↔ Prozessor ↔ Hauptspeicher)
- Je Datenstrom und Instruktionsstrom zwei Klassen:
 - eins (single) oder viele (multiple)
 - Duale Unterscheidungsmerkmale

Source: Flynn, Michael J.; , "Some Computer Organizations and Their Effectiveness," *IEEE Transactions on Computers*, vol.C-21, no.9, pp.948-960, Sept. 1972

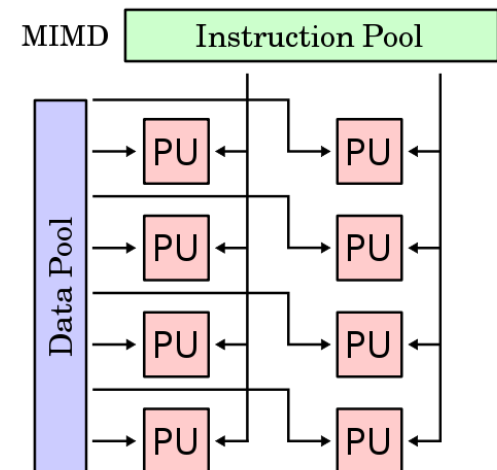
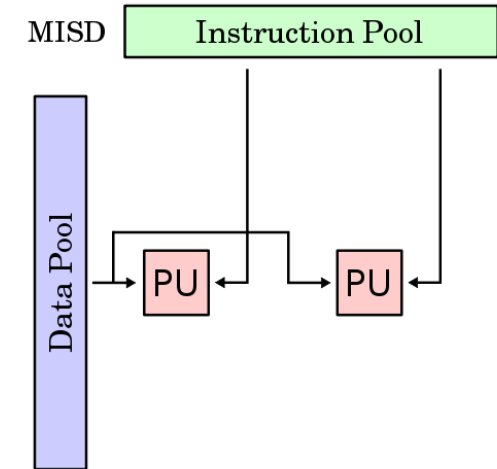
Flynn'sche Taxonomie – Single Instruction

- Single Instruction, Single Data (SISD)
 - Von-Neumann-Architektur
 - gemeinsamer Daten- und Instruktions-Speicher
 - Harvard-Architektur
 - getrennter Daten- und Instruktions-Speicher
- Single Instruction, Multiple Data (SIMD)
 - Vektorprozessor
 - schnelle Ausführung gleichartiger Rechenoperationen auf mehrere gleichzeitig eintreffende oder zur Verfügung stehende Eingangsdatenströme



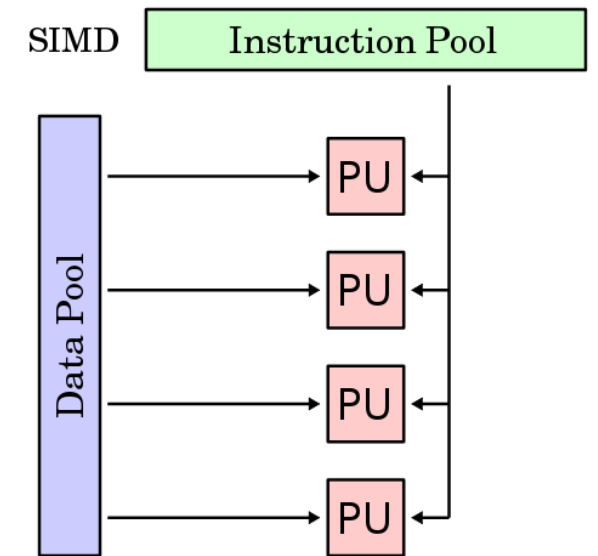
Flynn'sche Taxonomie - Multiple Instruction

- Multiple Instruction, Single Data (MISD)
 - Fehlertolerante, redundante Rechner
 - Redundante Datenströme zur Fehlererkennung bzw. -korrektur
- Multiple Instruction, Multiple Data (MIMD)
 - Multiprozessorsysteme
 - Führen gleichzeitig verschiedene Operationen auf verschiedenen gearteten Eingangsdatenströmen durch
 - Nebenläufige Kontrollflüsse
 - Kopplung über Verbindungs-Netzwerke



Vector Processor

- Eine Instruktion wird auf mehreren unabhängigen Operanden ausgeführt
- Beispiel
 - Skalarprodukt von zwei Vektoren: $\vec{x}\vec{y} = \sum_{i=1}^N x_i y_i$
 - Produkt wird parallel auf mehreren ALUs berechnet
- Wurde zuvor hauptsächlich in wissenschaftlichen Bereichen genutzt und war daher nur in Supercomputern zu finden (Beispiel: Cray-1)



SIMD - Instruktionen

- Multimedia Anwendungen
 - Graustufen (1 byte), Volumenwerte (2 bytes), Farbwerte (3 bytes)
- Viele kleine unabhängige Datenelemente werden gleichzeitig verarbeitet
- Viele Optionen für Parallelität
- 32-bit Register und die 32-bit ALU des Prozessors werden nicht vollständig ausgenutzt
- SIMD auf packed data
 - Reduziert die Anzahl an Taktzyklen für die selbe Menge an Daten
 - Vektorisierung von Daten und parallele Berechnung

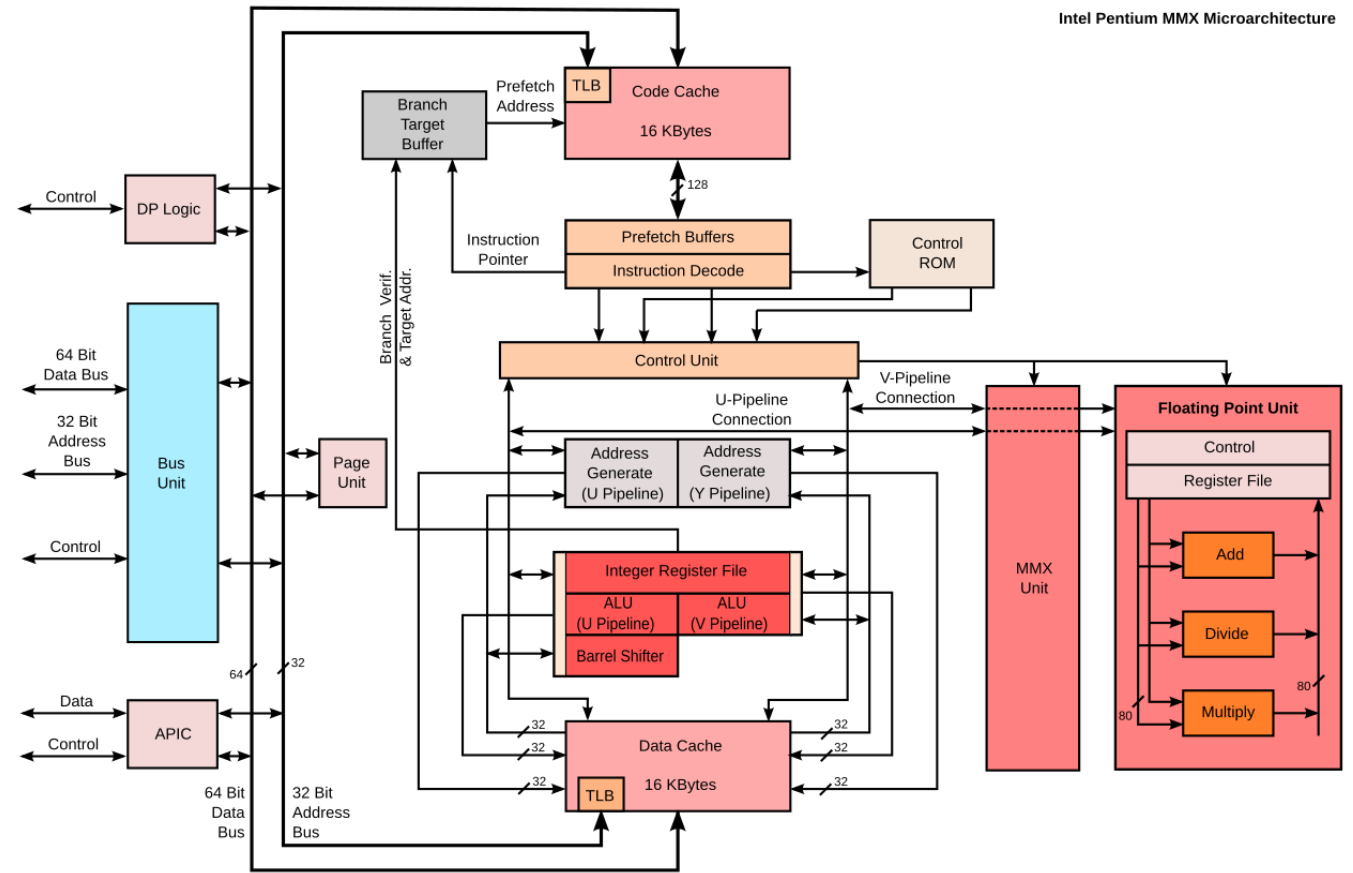
Beispiel – Multi Media Extension (MMX)

- SIMD Erweiterung des IA-32 Instruktionssatzes
- FPU Register (80 bits) werden genutzt
 - 64-Bit-Register MM0 - MM7
- 24 neue Instruktionen um ganzzahlige Werte zu verarbeiten
 - Arithmetik, Vergleiche, Konvertierung, logische Operationen, Shifts, Transfer
- Vier neue Vektor Daten Formate



Intel Pentium MMX Microarchitektur

- Erster superskalärer x86 Prozessor
 - Duales Integer Pipeline Design
 - Main pipe (U) kann jede Art von Instruktion verarbeiten
 - Pipe (V) kann die gängigsten einfachen Instruktionen verarbeiten
- Advanced floating-point unit
- MMX konnte nicht gleichzeitig mit x87 FPU Instruktionen genutzt werden

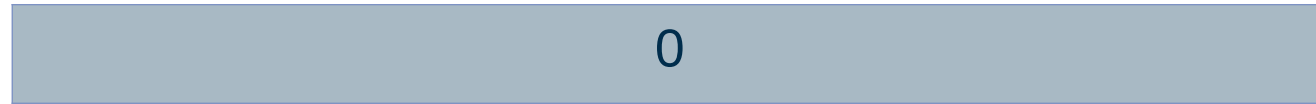


Source: Intel Pentium MMX Arch, [Wikimedia](#)

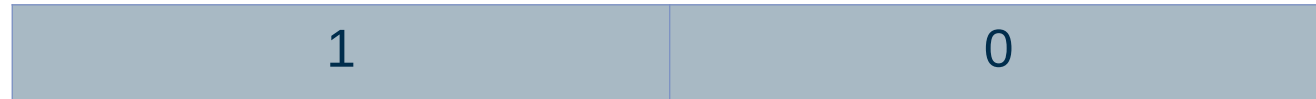
SIMD – Sub-Word Execution Model (I)

- Beispiel: teile 64-Bit Datentyp in kleinere Einheiten
- Instruktionen werden auf kleineren execution units / modified units ausgeführt

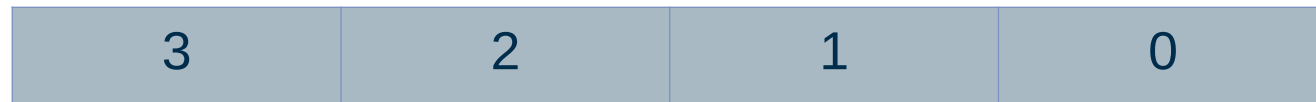
- Quad Word



- Packed Double Word



- Packed Word



- Packed Byte



SIMD – Sub-Word Execution Model (II)

- Beispiel 1: Addition
 - Selbe Instruktion parallel auf 4 execution units / modified unit
- `ADD R3 := R1, R2`

■ R1

a_3	a_2	a_1	a_0
-------	-------	-------	-------

+

+

+

+

■ R2

b_3	b_2	b_1	b_0
-------	-------	-------	-------

=

=

=

=

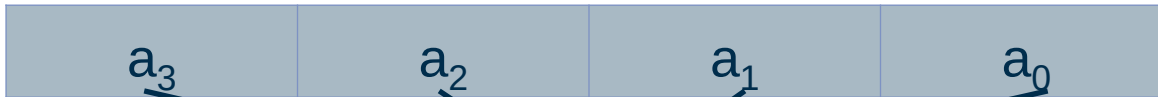
■ R3

a_3+b_3	a_2+b_2	a_1+b_1	a_0+b_0
-----------	-----------	-----------	-----------

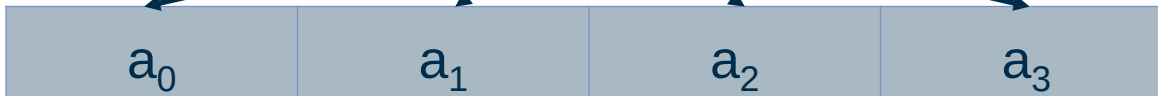
SIMD – Sub-Word Execution Model (III)

- Beispiel 2: Permutation
 - Permutation von half-word Reihenfolge
- PERMUTE R3 := R1 (Pattern 0123)

■ R1

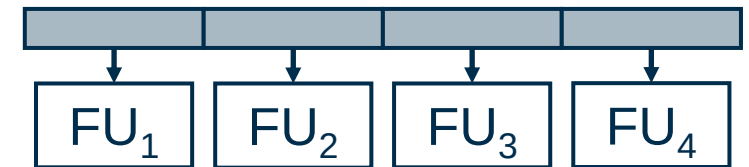
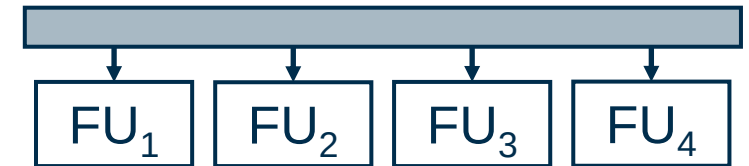


■ R3



VLIW Instruction vs. SIMD Instruktionen

- SIMD Instruktionen sind multimedia Instruktionen in denen ein Opcode eine ähnliche Operation auf mehreren Operanden (-paaren) ausführt
- In einem VLIW Paket sind typischerweise verschiedene Arten von Instruktionen



Übersicht – Vorlesung Nr. 03

- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Speicherhierarchien
 - Überblick Technologien

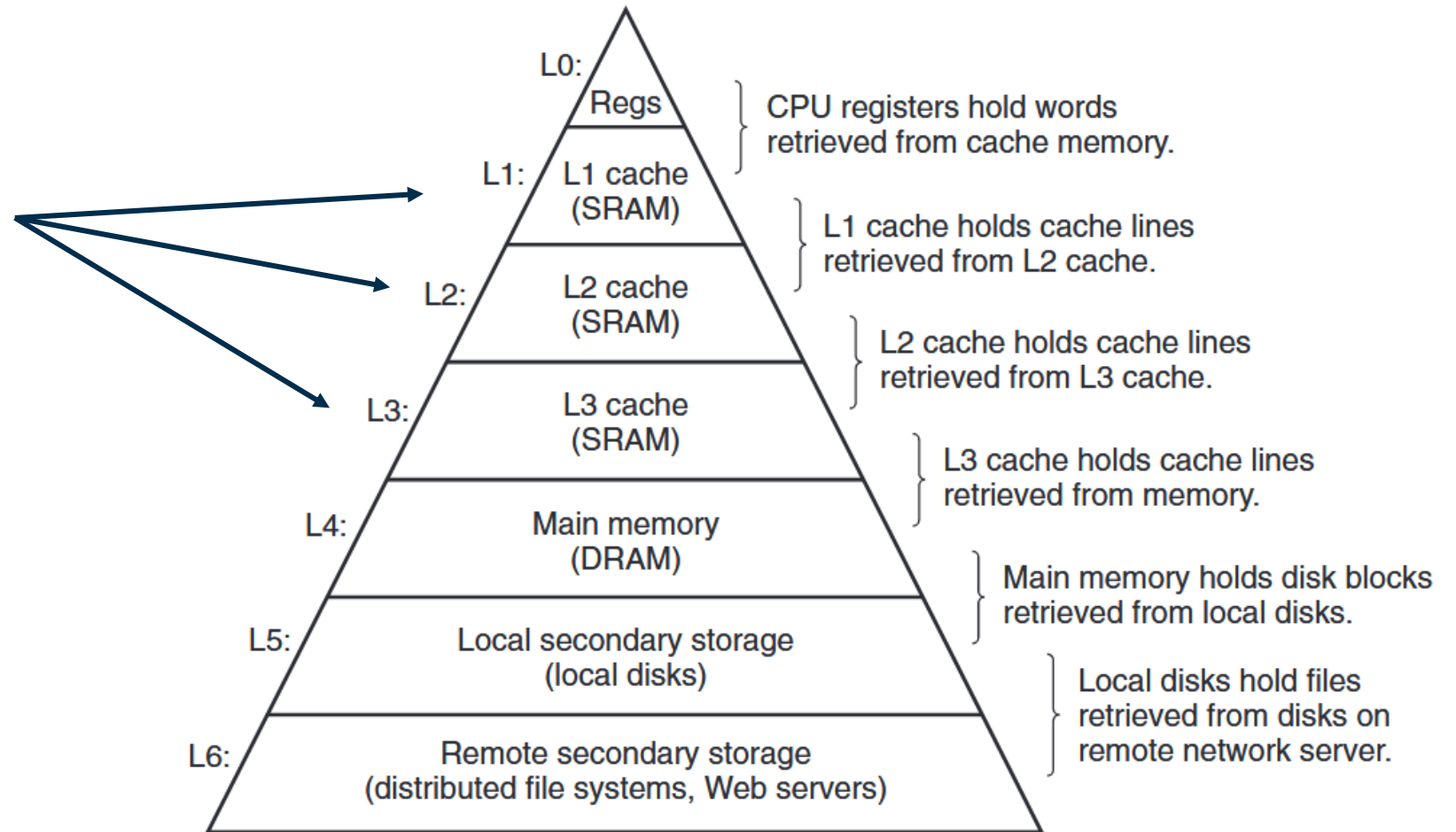
Caches

- Caches reduzieren unperformante Hauptspeicherzugriffe, durch Einsatz schnellerer Zwischenspeicher (Hauptspeicher ist 15 x langsamer)
- Cache kommt aus dem Französischen: cacher (verstecken)
- Das Cache ist software-transparent, d.h. der Benutzer braucht nichts von seiner Existenz zu wissen.



Speicherhierarchie

- Zwischen den Registern und dem Arbeitsspeicher liegen mehrere Caches
- Die Caches bieten schnelleren Speicher, der vielbenutzte Daten vom langsameren Speicher hält



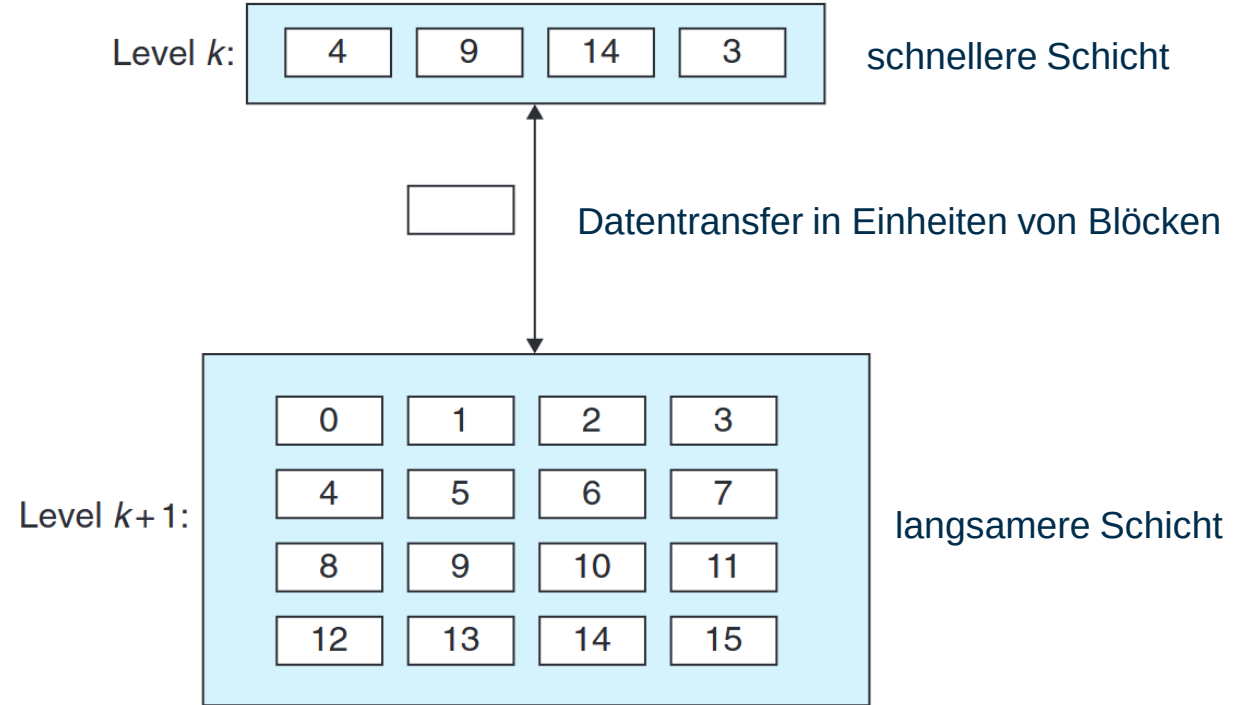
[1] S.44

Speicherhierarchie und Performanz

Memory Type	Access Time	relative
Processor Register	100 ps to 1 ns	0,005
Processor Cache L1	1 ns to 5 ns	0,05
Processor Cache L2	2 ns to 20 ns	0,1
Processor Cache L3	30 ns to 50 ns	0,4
Main Memory (DRAM, DDR-SDRAM)	50 ns to 200 ns	1
Mass Storage (Hard Disk, Solid State Disk)	8 ms to 10 ms (0,5 ms to 1 ms)	50
Removable Media (Tape, DVD, USB-Stick)	seconds to minutes	>1000

Caches

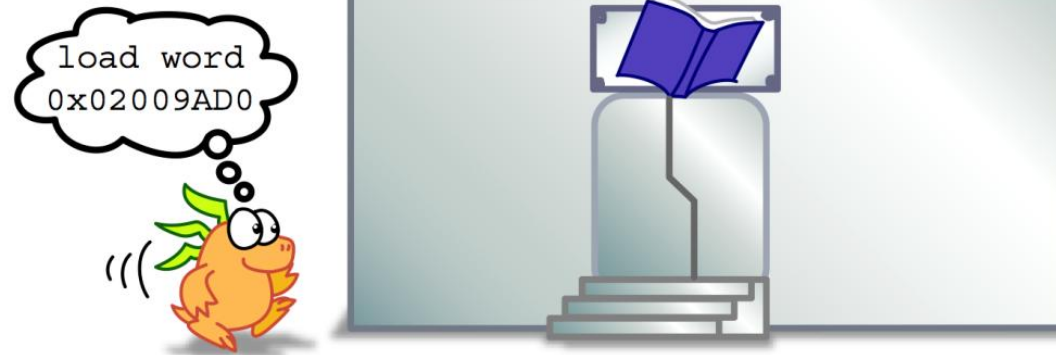
- Daten werden in Blöcken gespeichert
- Wenn Daten gebraucht werden, wird zunächst in der schnelleren Schicht geschaut



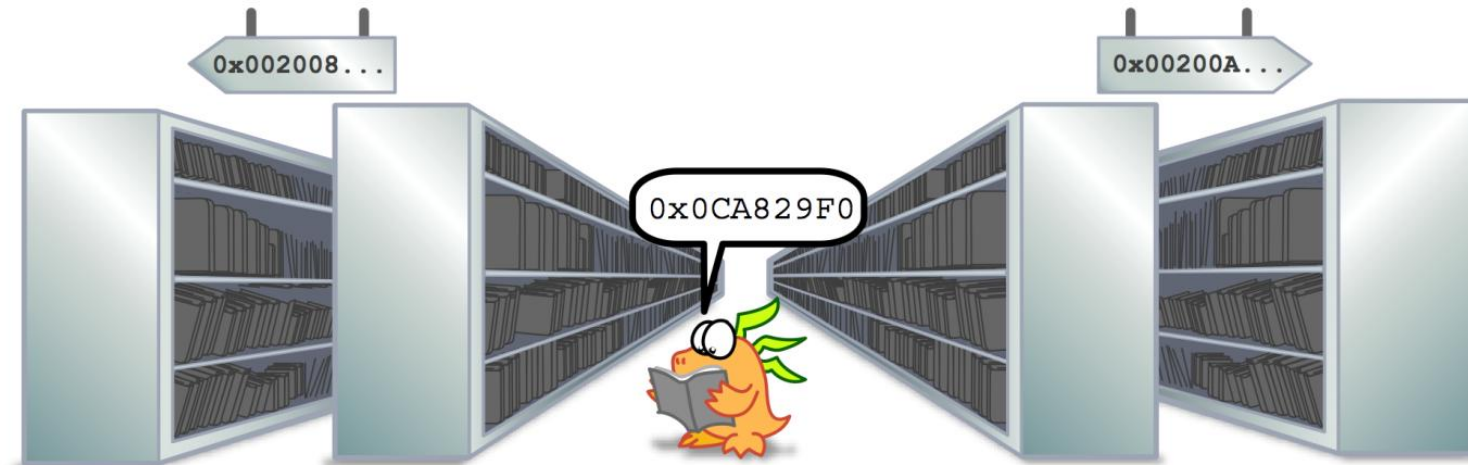
[1] S.639

Caches

For computers, memory accesses are like going to the library,



Finding the necessary information in the page of a book,



Source: University of California, Berkeley, [CS illustrated](#)

Caches

And going back home to do the work involving that information.



Hurry up, will ya?!



While computers don't mind going back and forth like this for data, it usually means users have to do a lot of waiting.



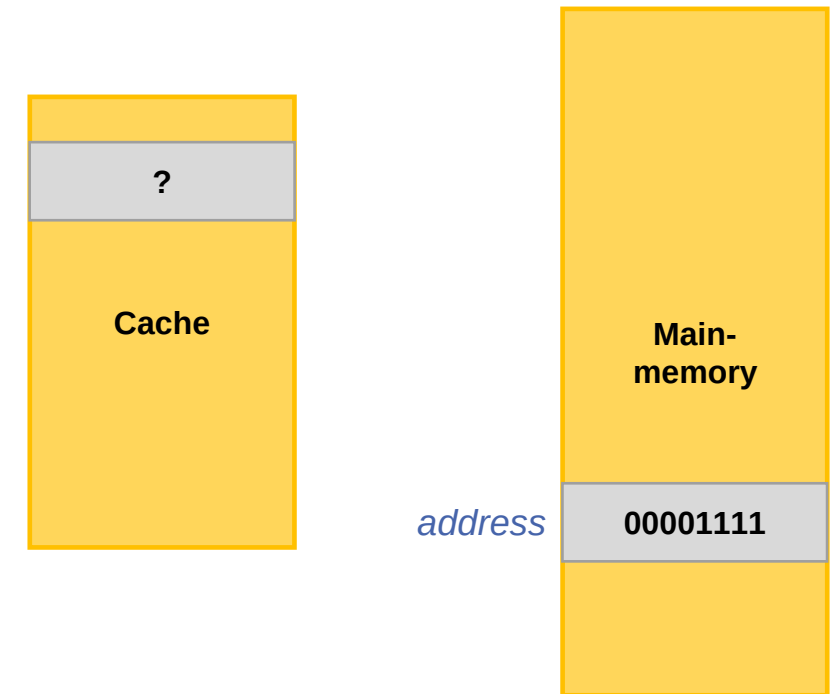
Fortunately for users, computers have caches, which is the equivalent of keeping copies of the books needed on a shelf near the workspace. Through a number of mechanisms, caches give the illusion of being able to access memory very quickly!



Source: University of California, Berkeley, [CS illustrated](#)

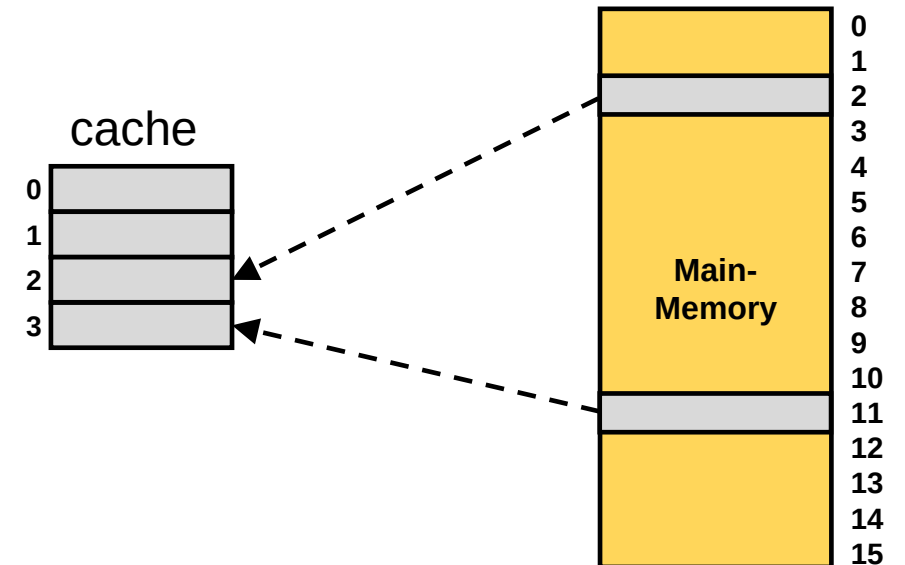
Caches - Lesezugriff

- CPU Load: Lese Datum aus dem Arbeitsspeicher unter Adresse *address*
- Kopie der Hauptspeicherzelle *address* im Cache
 - Falls ja (**cache hit**)
 - so entnimmt die CPU das Datum aus dem Cache. Die Überprüfung und das eigentliche Lesen aus dem Cache erfolgt in einem Zyklus, ohne ein Wartezyklus einfügen zu müssen.
 - Falls nein (**cache miss**)
 - Zugriff auf den Arbeitsspeicher, laden der Cachezeile des Datums in den Cache und laden des Datums von dort in die CPU



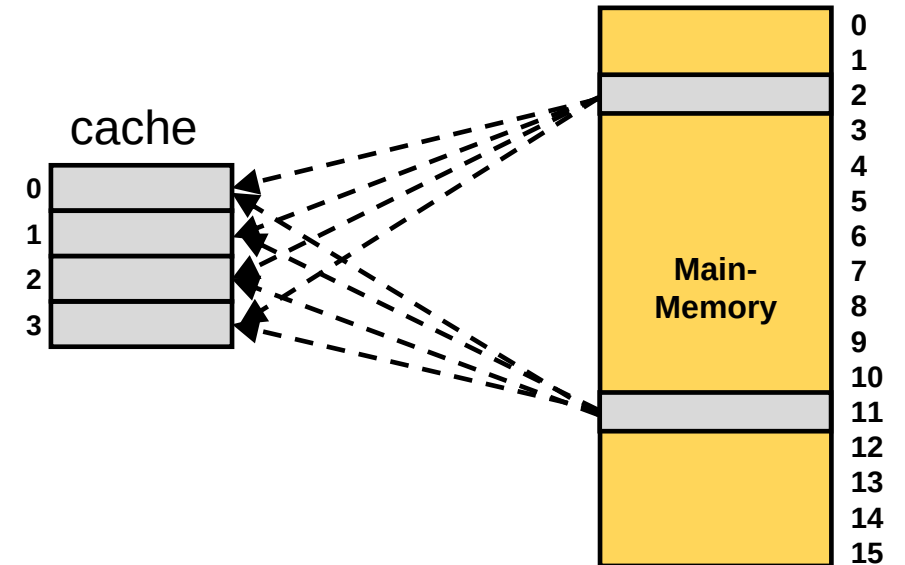
Caches - Organisation - Direct Mapped

- Jede Adresse ist nach einem festen Schema (z.B. basierend auf einem Teil der Adresse) direkt einer Cachezeile zugeordnet.
- Einfach, aber niedrige Hit-Rate
- Nachteil keine Flexibilität: Verdrängung kann auftreten wenn der Cache nicht voll ist.



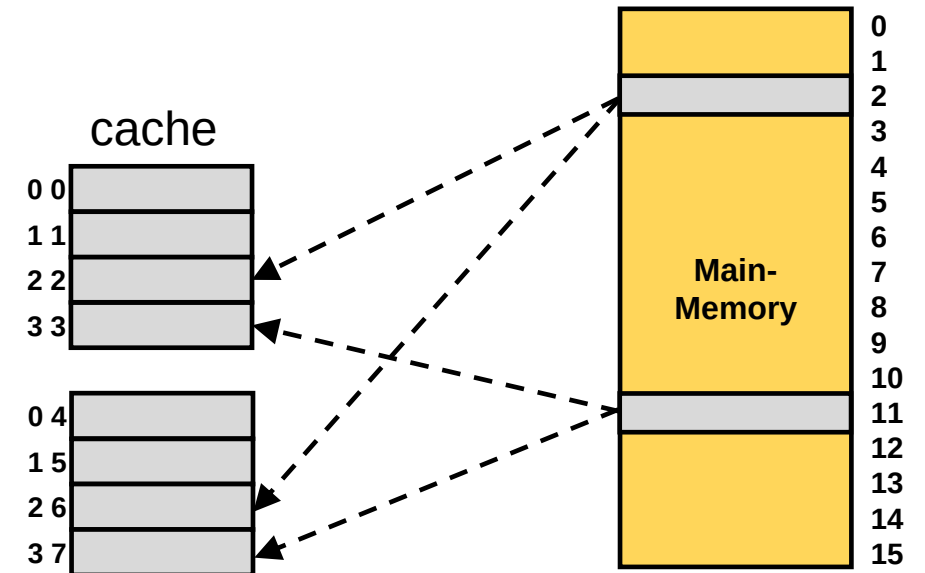
Caches - Organisation - Voll-Assoziativ

- Jeder Block kann in jeder Cachezeile gespeichert werden.
- Sehr hohe Hit-Rate
- Nachteil: für das Auffinden eines Blocks im Cache muss das Tag jeder Zeile verglichen werden (hoher Aufwand, komplex)

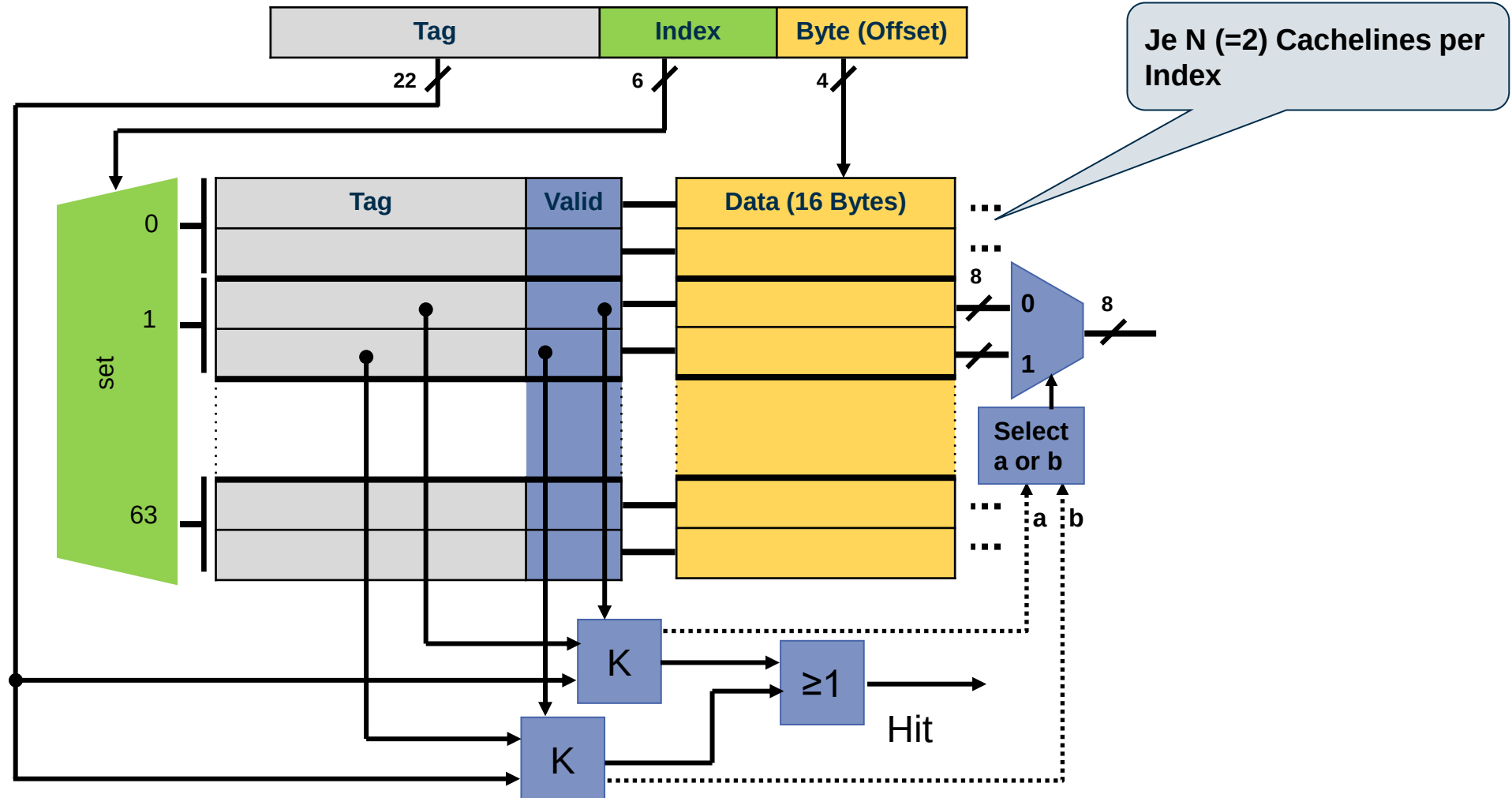


Caches - Organisation - n-Wege Assoziativ

- Kombination aus Voll-Assoziativ und direct-mapped Cache
- Speicheradresse wird in 1 Set des Caches gemapped
 - Set besteht aus mehreren Cache Lines
 - Vergleich nur Innerhalb des Sets
- Vergleichbar zu n unabhängigen Caches
- Beispiel: 2 Wege Assoziativ, jeder Index hat zwei Möglichkeiten



Caches – 2-Wege assoziativer Cache



Cache miss, heiße und kalte Caches

- Heißer Cache: Cache arbeitet optimal, nur wenige Misses
- Kalter Cache: hohe Missrate
- Es werden drei Arten von Cache Misses unterschieden:

- **Compulsory**

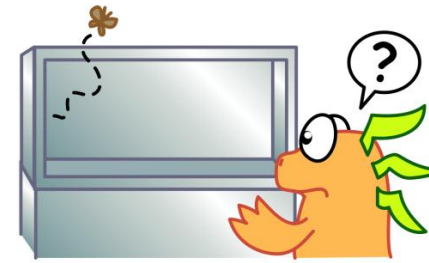
- Beim erstmaligen Zugriff befinden sich die Daten noch nicht im Cache.
- Abhilfe: Spekulatives Laden durch Prefetcher

- **Capacity**

- Daten wurden verdrängt, weil der Cache zu klein ist.
- Abhilfe: Vergrößerung des Caches

- **Conflict**

- In einem Set ist nicht ausreichend Platz obwohl andere Sets frei sind.
- Abhilfe: Erhöhung der Assoziativität



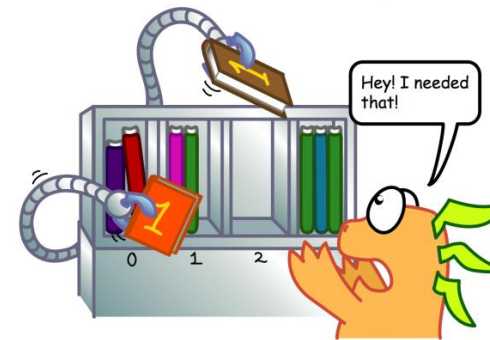
Compulsory

Compulsory misses happen when a block is referenced for the first time. The computer can't get a block that doesn't exist yet!



Capacity

The block is not in the cache because there is no space in the cache for it. Caches are of finite size, after all.



Conflict

These types of misses happen only in direct-mapped and set-associative caches. Multiple blocks can be mapped to a set, forcing evictions when the set is full.

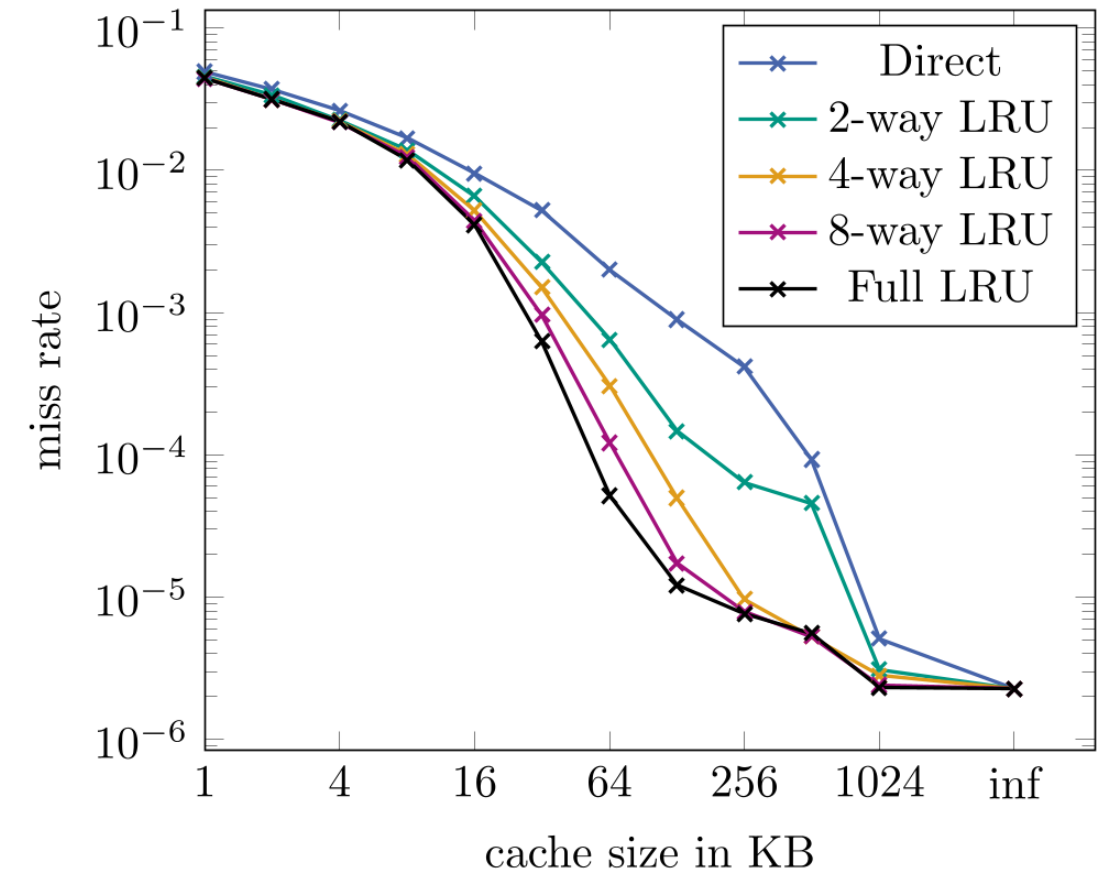
<http://csillustrated.berkeley.edu/PDFs/handouts/>

Caches - Verdrängungsstrategien

- FIFO (First-In-First-Out):
 - schlechte Strategie, da der Block, der zuerst geladen wurde, als erstes überschrieben wird
 - Blöcke werden überschrieben, die häufig gebraucht werden, z.B. während Initialisierung
- Optimale Ersetzungsstrategie:
 - der Block, dessen Daten in Zukunft am längsten (zeitlich) nicht mehr angesprochen werden, wird ausgelagert
 - Nachteil: diese Information steht meist nicht zur Verfügung
- LRU (Least-Recently-Used):
 - der Block, der in der Vergangenheit am längsten (zeitlich) nicht mehr gebraucht wurde, wird ausgelagert
- Random
 - Zufällige Auswahl des Blockes, der ersetzt wird

Caches – Miss Rate Comparison

- Cache Performance for Spec CPU2000 Benchmark
- Source: University of Wisconsin–Madison, Multifacet Research Group
- Miss rate
 - Number of cache misses / number of requests
- Der arithmetische Mittelwert wird über alle Benchmarks hinweg berechnet
- Die Berechnung erfolgt für die ersten 1 Milliarde Befehle jedes Benchmarks



Übersicht – Vorlesung Nr. 03

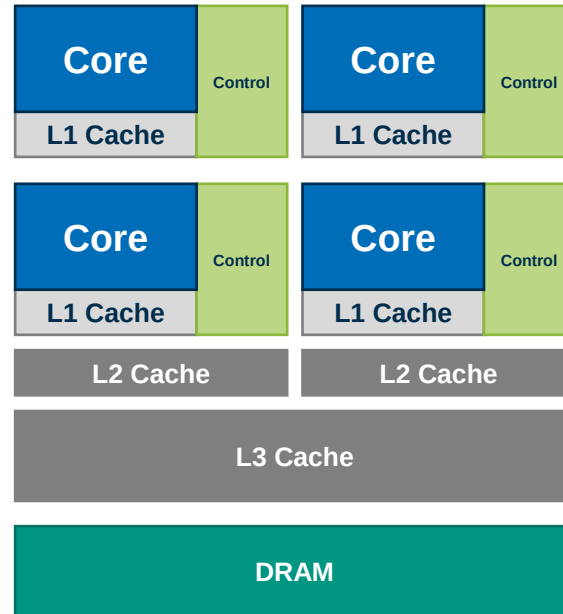
- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Speicherhierarchien
 - Überblick Technologien

Motivation Graphics Processing Unit (GPU)

- Herausforderung: Echtzeit Grafik erfordert die Berechnung von Millionen von Pixeln mehrmals pro Sekunde
 - z.B., 4K Display = $8.3\text{M Pixel} \times 60\text{ fps} = 500\text{M Pixel Berechnungen/s}$
- Maximiere Parallelität
 - Die Berechnungen der Pixel sind überwiegend unabhängig
- Spezialisierte Hardware kann diese Durchsatz-orientierten Aufgaben viel effizienter berechnen als eine CPU mit wenigen komplexen Cores
- Matrix und Vektor Operationen in neuronalen Netzen sind ebenfalls unabhängig und somit profitieren KI Berechnungen von derselben Parallelität

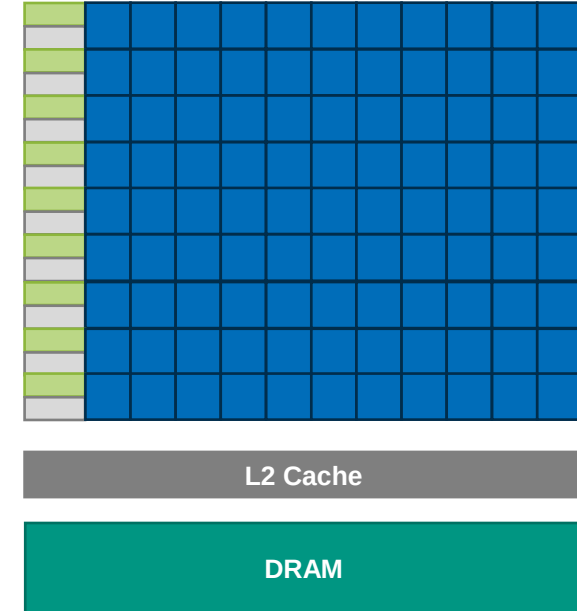
CPU vs. GPU

- Tiefe Dichte
- Komplexe Kontrolleinheit
- Große Caches



- Daten Caching und Flow Control
- Optimiert für Sequenz von Operationen
 - Führt ein paar dutzend Threads parallel aus

- Hohe Dichte
- Viele Berechnungen pro Speicherzugriff
- Viele parallele execution units



- Fokus auf Datenverarbeitung
- Hoher Instruktionsdurchsatz
 - Führt tausende Instruktionen parallel aus

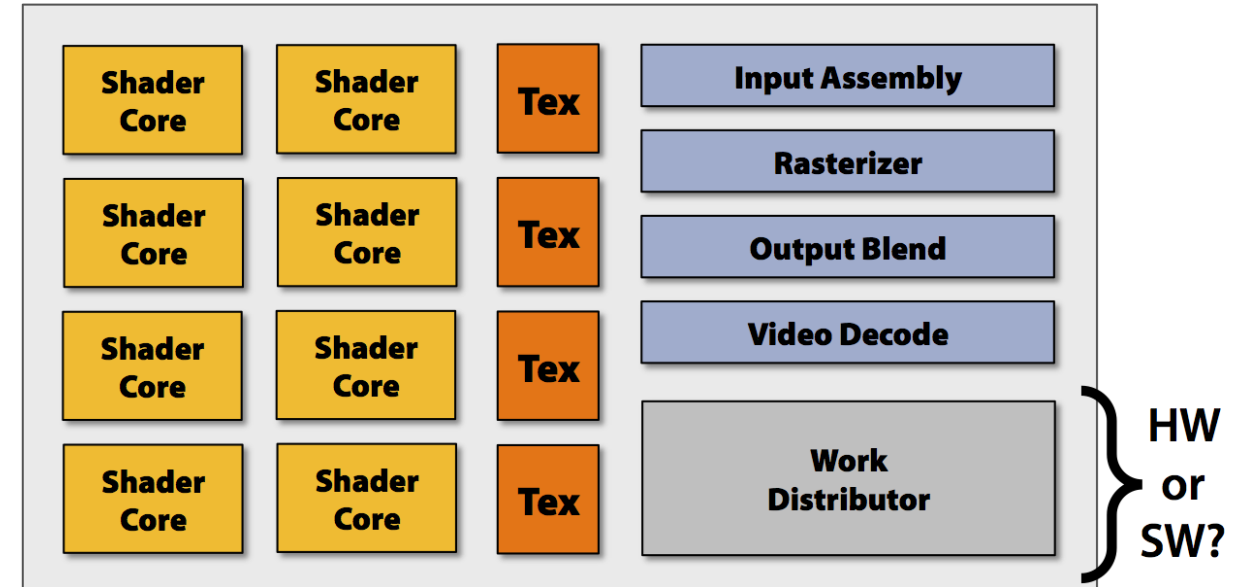
Source https://docs.nvidia.com/cuda/archive/11.2.0/pdf/CUDA_C_Programming_Guide.pdf

CPU vs. GPU

- CPUs können kompliziertere Workflows als GPUs ausführen
- CPU haben nicht so viele ALUs oder FPUs wie GPUs
 - die ALUs and FPUs in einem CPU Core sind jeweils mächtiger
- CPUs haben mehr Cachespeicher als GPUs
- GPUs wurden für hoch-parallelisierbare Aufgaben entworfen
- In der Regel enthalten Anwendungen eine Mischung aus parallelen und sequenziellen Teilen
 - Systeme werden für gemischte Nutzung von GPUs und CPUs entworfen

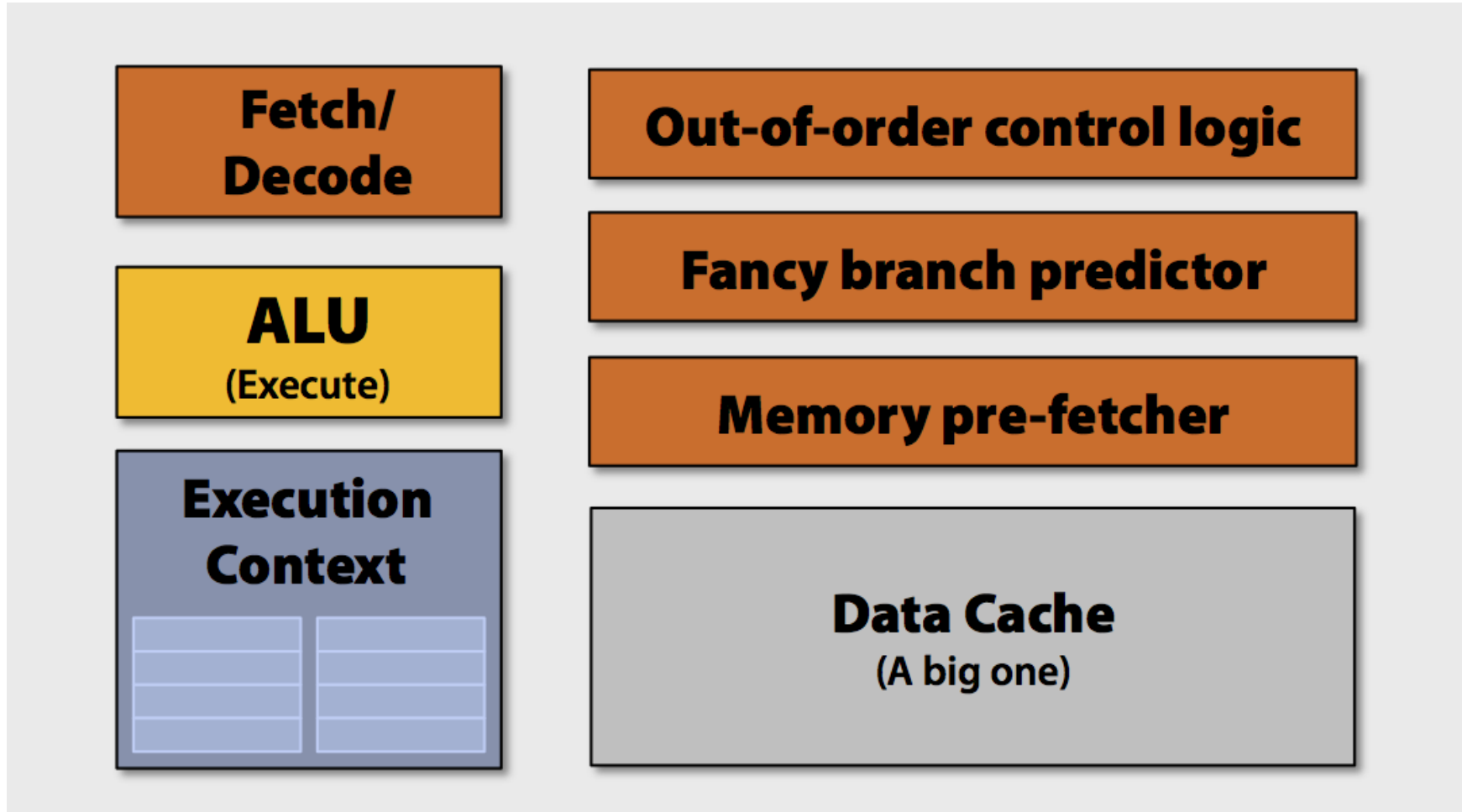
Woraus besteht eine GPU?

- Heterogener Chip Multi-Prozessor
 - hoch spezialisiert für Grafik (parallele, floating-point intensive Aufgaben)
- Kernkomponenten:
 - Tausende einfache Cores werden in Gruppen organisiert
 - Spezialisierte Speicherhierarchie (Register, gemeinsam, global)
 - Hardware scheduler für enorme Parallelität



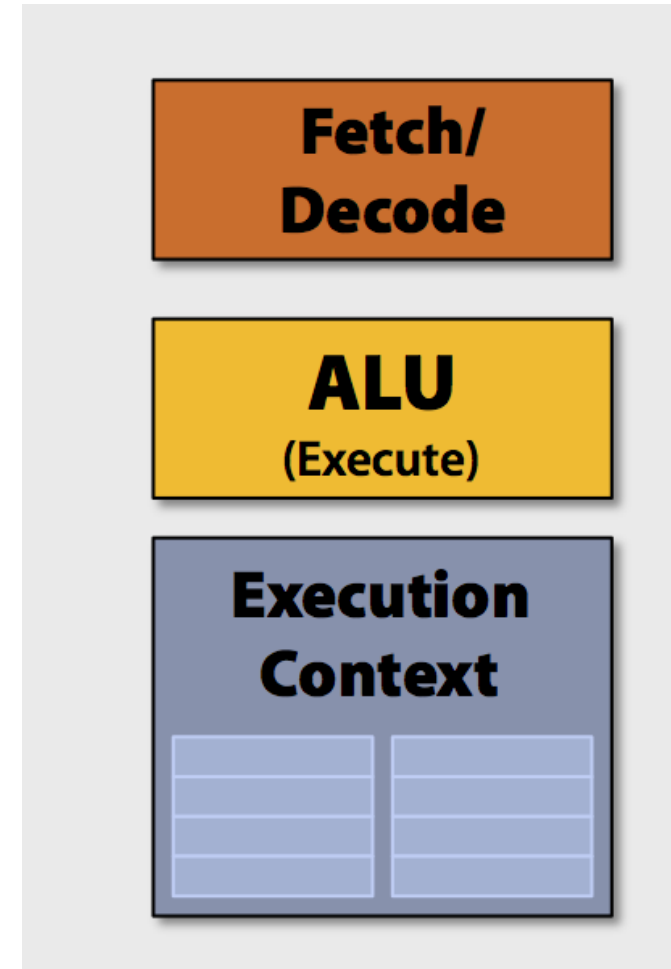
Source: fatahalian-gpu-architecture.pdf

CPU-Style Cores (I)



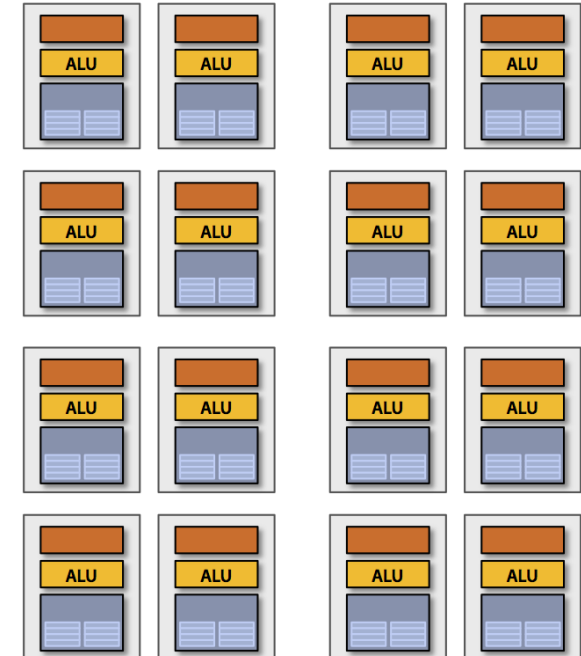
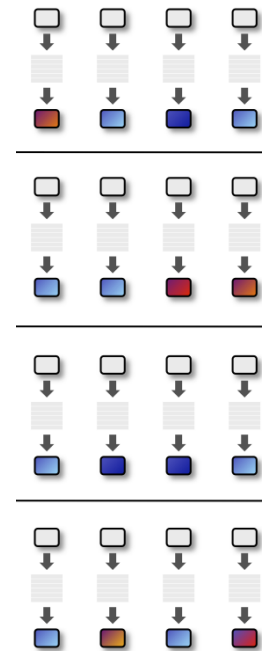
CPU-Style Cores (I)

- Idee #1:
 - Komponenten entfernen, die für die Beschleunigung einzelner Instruktionen optimiert wurden



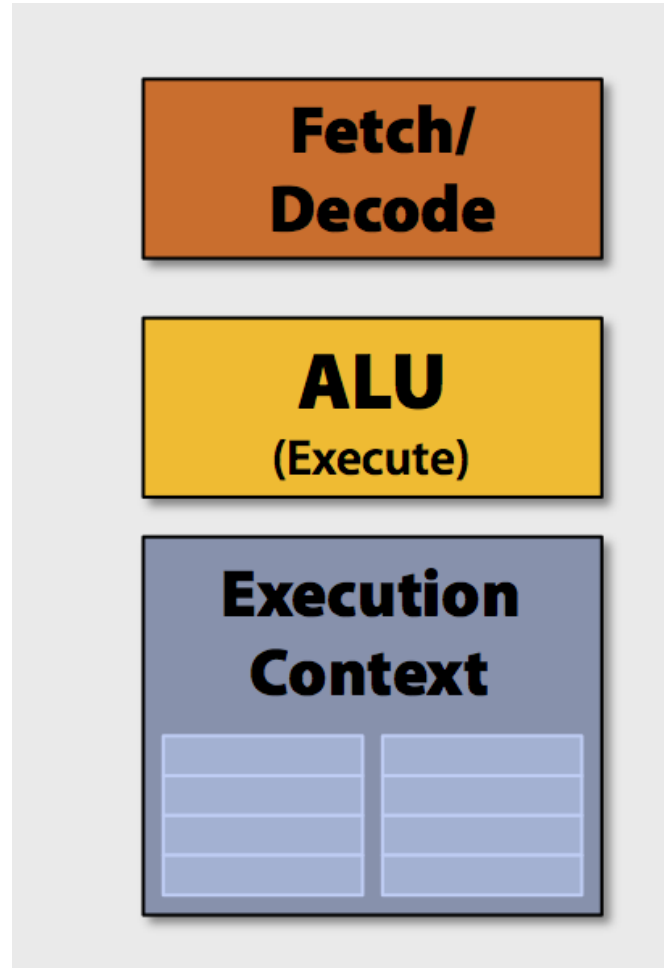
CPU-Style Cores (II)

- 2 cores
 - 2 Fragmente parallel
 - 2 simultane Instruktionsströme
- 4 cores
 - 4 Fragmente parallel
 - 4 simultane Instruktionsströme
- 16 cores
 - 16 Fragmente parallel
 - 16 simultane Instruktionsströme



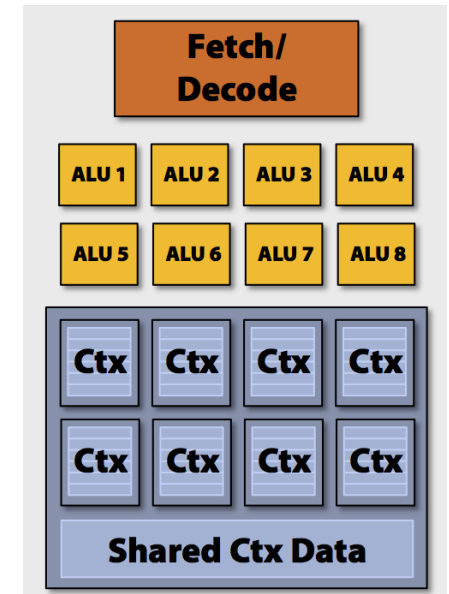
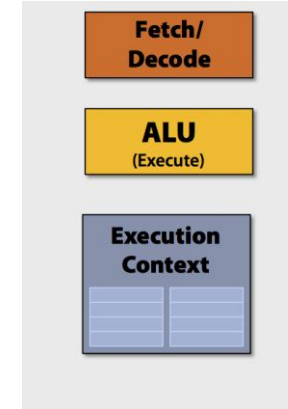
Single Processor Core

- Nur... Wie viele Fragmente sollen sich einen Instruktionsstrom teilen?



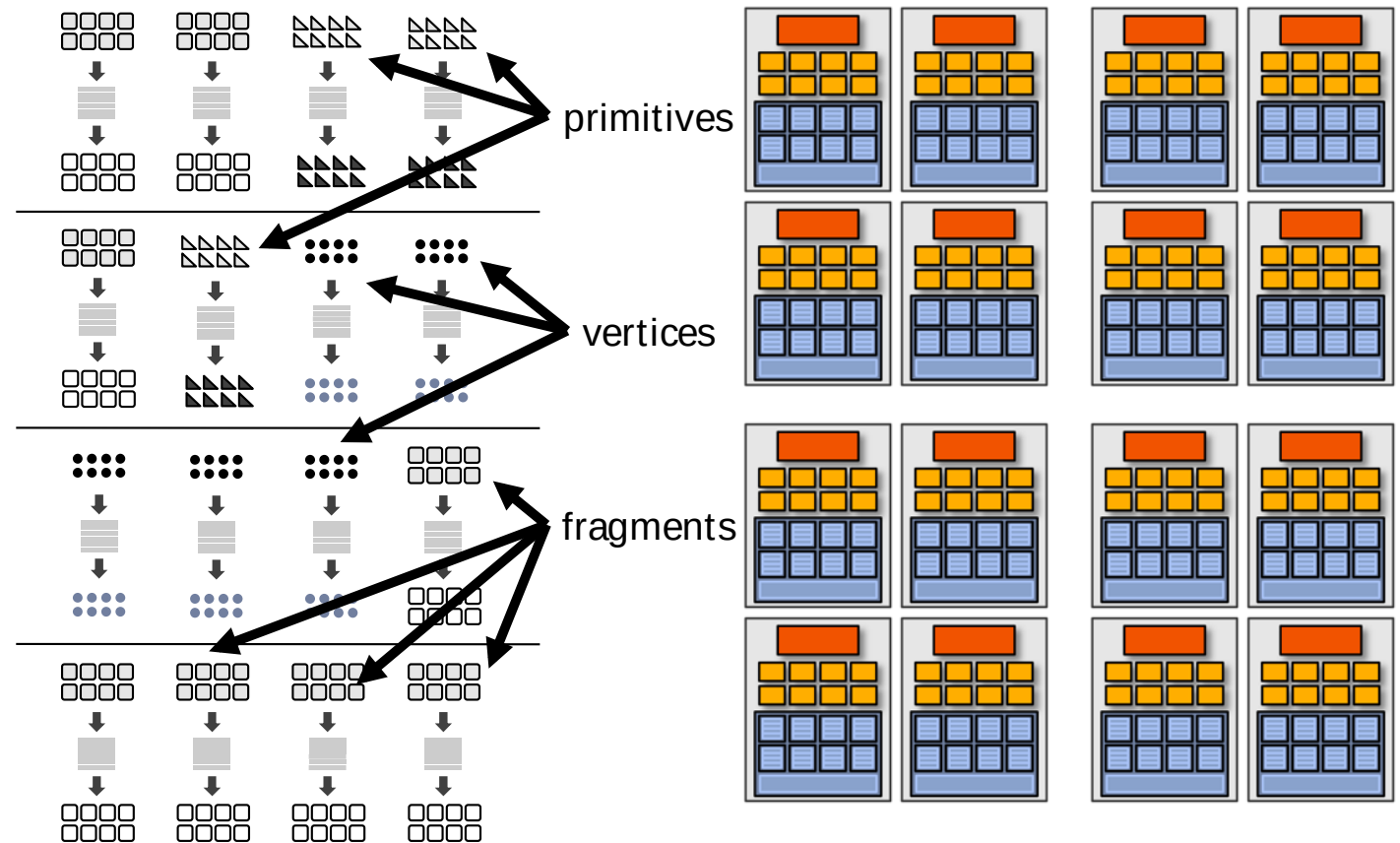
Von Single ALU zu SIMD Verarbeitung

- Idee #2:
 - Amortisiere Kosten/Komplexität der Verwaltung eines Instruktions Streams über mehrere ALUs
- Füge viele ALUs hinzu → SIMD processing
 - Neu kompilierter Shader sollte auf die Anzahl an ALUs im Shader Core angepasst werden
 - Hier: 8 parallele Fragmente für 8 ALUs
- Shader Core repräsentiert Menge von ALUs mit geteiltem Instruktions Fetch/Decode



128 [Vertices/fragments Primitives CUDA threads OpenCL work items Computer shader threads] parallel

- 16 Shader Cores mit je 8 internen Fragmenten (ALUs)
 - Insgesamt 128 Fragmente die parallel genutzt werden können
 - Weiterhin 16 gleichzeitige Instruktions Streams



SIMD Processing

- SIMD Verarbeitung impliziert nicht SIMD Instruktionen
- Option 1: Explizite Vektor Instruktionen
 - Intel/AMD x86 SSE, Intel Larrabee, Intel Xeon Phi
- Option 2: Skalare Instruktionen, implizite HW Vectorisierung
 - HW legt fest wie Instruction Stream zwischen ALUs geteilt wird (genaue Aufteilung nicht sichtbar für Software)
 - NVIDIA GeForce (“SIMT” warps), AMD Radeon Architekturen

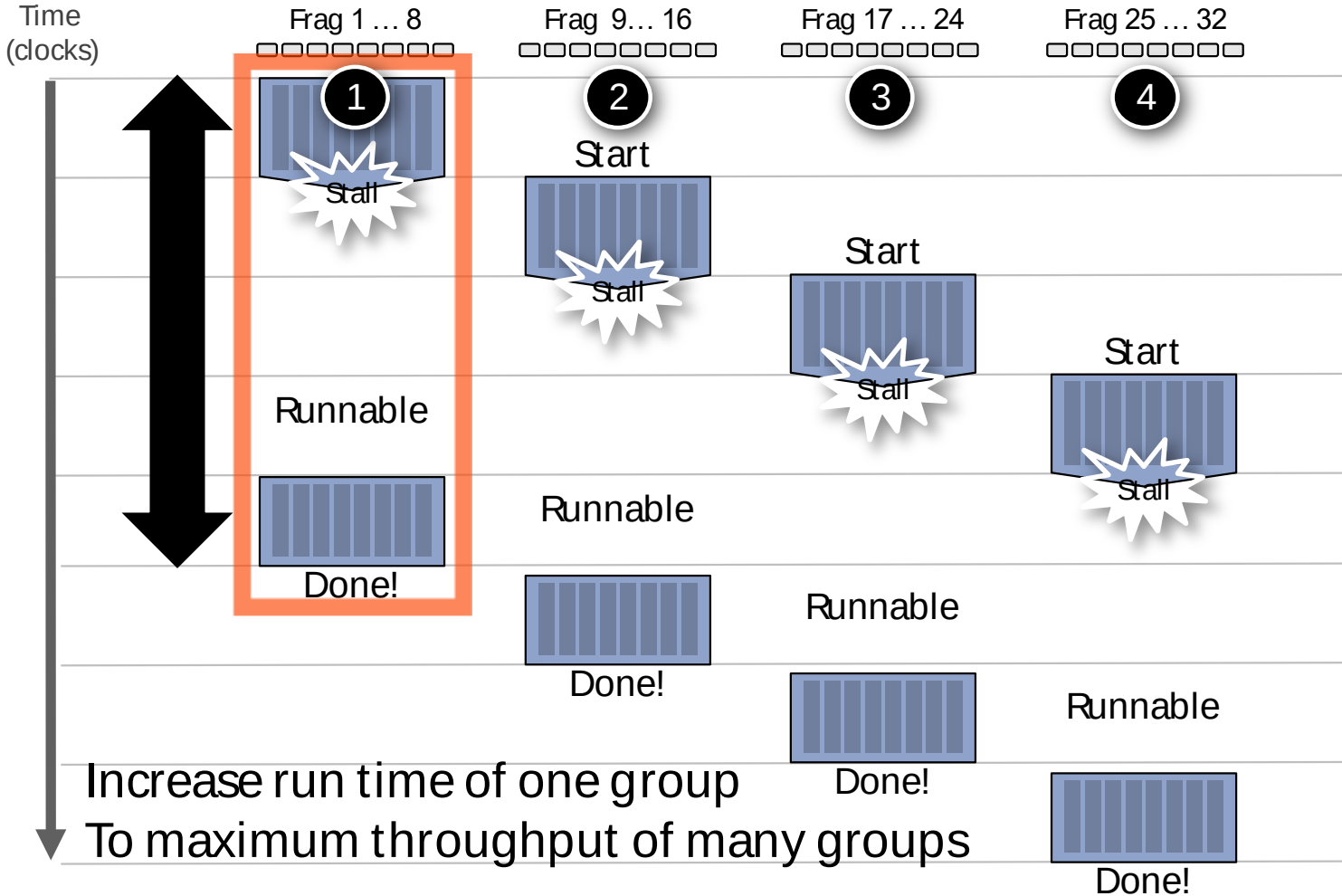


- In der Praxis: 16 bis 64 Fragmente teilen sich einen Instruktionsstrom

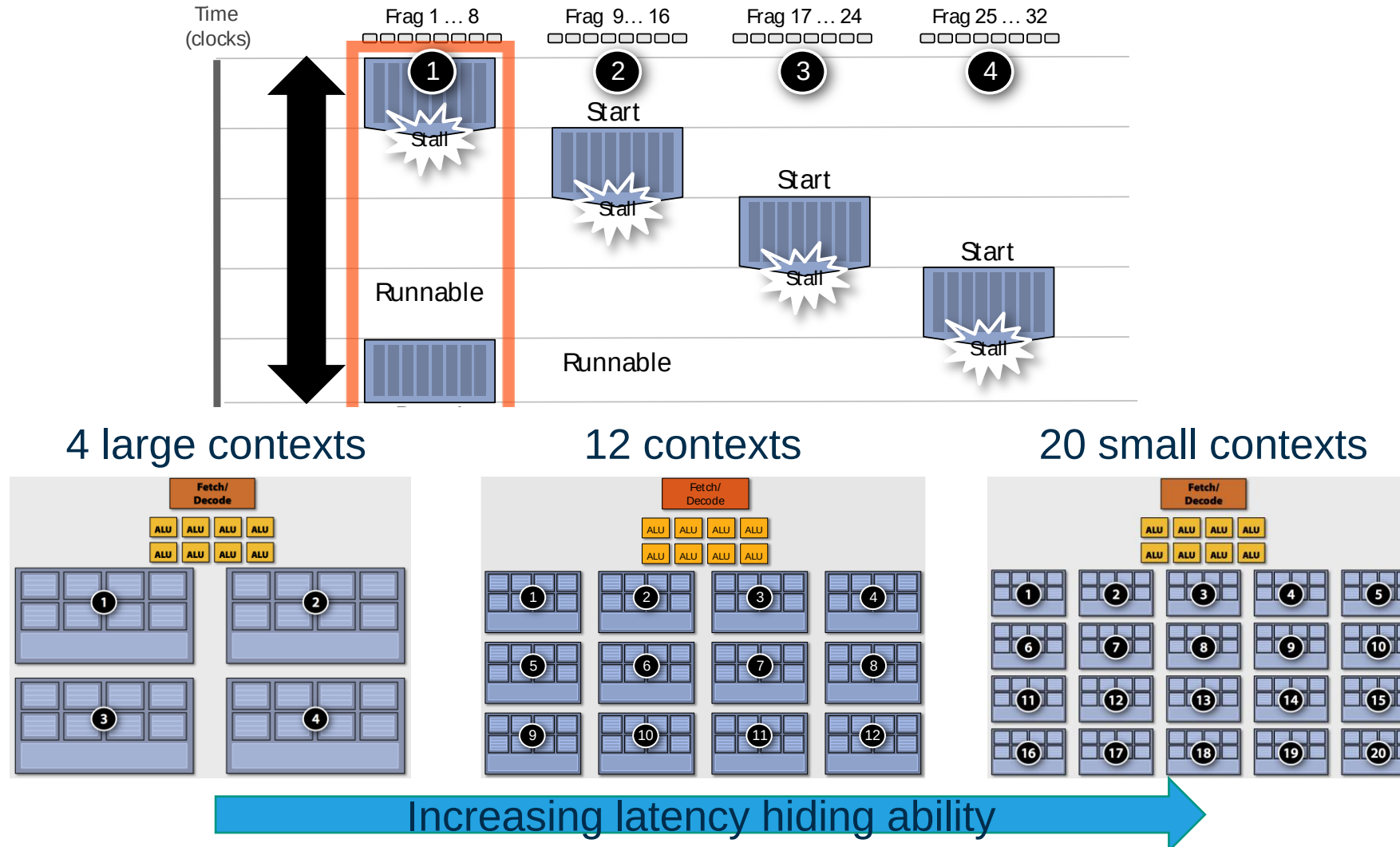
Stalls und Interleaves

- Stalls kommen vor, wenn ein Kern aufgrund einer Abhängigkeit (z.B. Datenabhängigkeit, Speicherzugriff) die nächste Instruktion nicht ausführen kann
 - Zugriffslatenzen im Bereich von 100en bis 1000en Zyklen
- Die “Fancy Caches und Logik” die normalerweise bei Stalls hilft, haben wir entfernt
- Aber wir haben eine Menge unabhängiger Fragmente
- Idee #3:
 - Die Ausführung vieler Fragmente verschachteln um Stalls durch Operationen mit hoher Latenz zu vermeiden.

Interleaved Processing (I)



Interleaved Processing (II)



Beispiel

16 cores

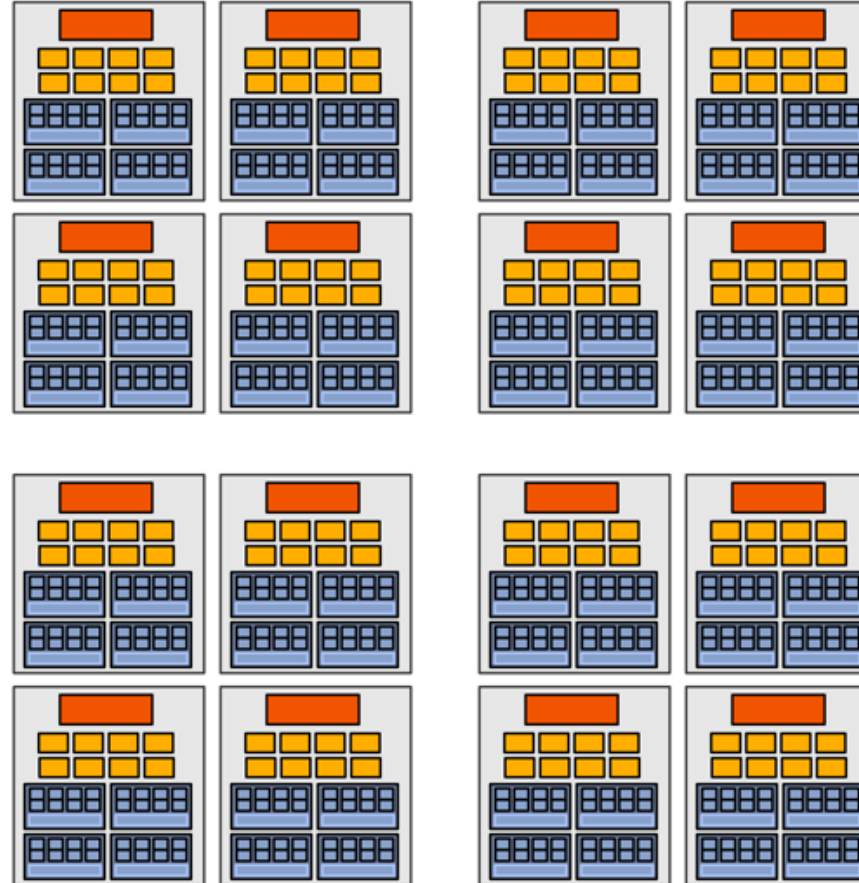
8 mul-add ALUs per core
(128 total)

16 simultaneous
instruction streams

64 concurrent (but interleaved)
instruction streams

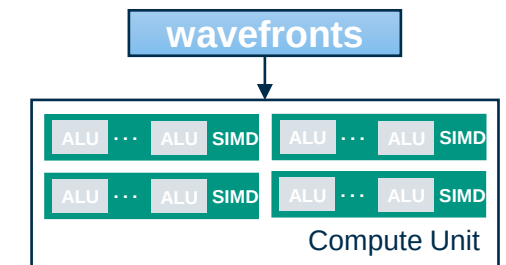
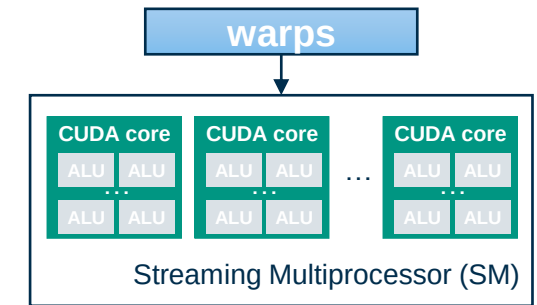
512 concurrent fragments

= 256 GFLOPs (@1GHz)



NVIDIA SIMT vs. AMD SIMD

- SIMT (Single Instruction, Multiple Thread)
 - NVIDIA Terminologie, Thread-orientiertes Programmiermodell
 - Jeder Thread hat eigenen Program Counter und Register
 - Alle 32 Threads einer warp führen gleichzeitig dieselbe Instruktion aus (lockstep)
- SIMD (Single Instruction, Multiple Data)
 - AMD Terminologie, traditioneller Vektorverarbeitungs Term
 - Vektor Register mit mehreren Datenwerten wird gleichzeitig verarbeitet
 - Fokus auf Parallelität auf Datenebene
 - 64 Threads pro wavefront
- Beide beschreiben dasselbe konzept
 - Gruppen von Threads führen gleichzeitig identische Instruktionen aus
 - Hardware koordiniert Instruktionsteilung automatisch

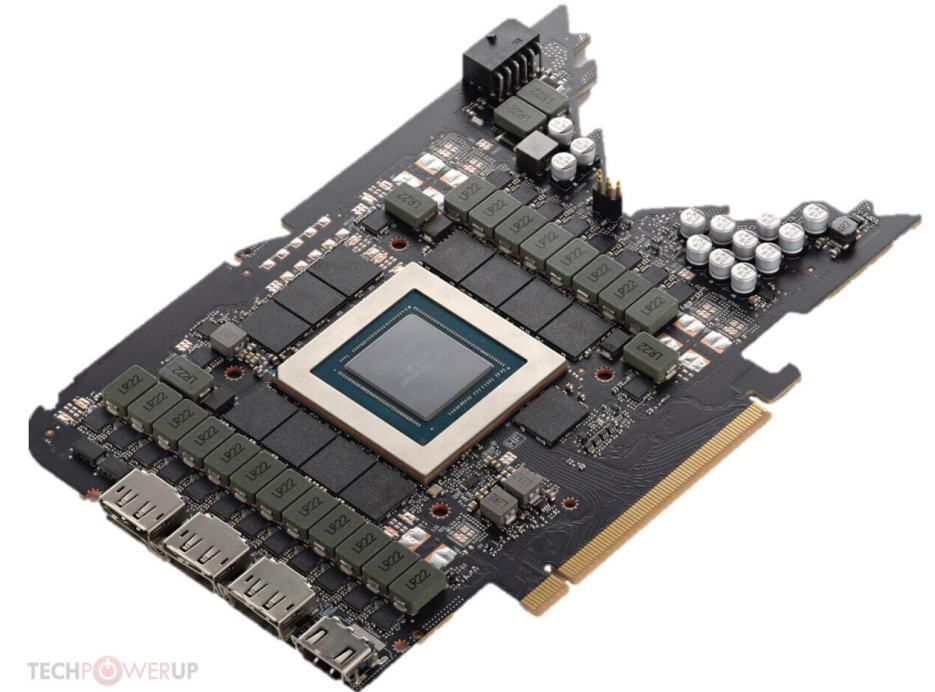


Hardware Einheiten: SMs & CUs

- NVIDIA: Streaming Multiprocessor (SM):
 - Grundlegende Recheneinheit einer GPU
 - Enthält CUDA Cores, Tensor Cores, Register Files pro warp, shared memory, warp scheduler
 - Hat gleichzeitig mehrere aktive warps → nicht fest
 - Tausende Threads möglich pro SM, welches dynamisch zwischen warps gewechselt werden
- AMD: Compute Unit (CU):
 - Äquivalent zu SM
 - Besteht aus SIMD ALUs, Großem Register File, local data share
 - Führt mehrere wavefronts parallel aus
 - Dynamisches Scheduling wie NVIDIA
- Ganze GPU = viele SMs/CUs

Beispiel: NVIDIA Ada Lovelace (AD102 “RTX 4090”)

- 144 SMs insgesamt (128 aktiviert auf RTX 4090)
- 128 CUDA Cores pro SM
- 16384 CUDA Cores insgesamt
- SIMT Ausführungsmodell, 32-lane warps
- 4 SIMD FUs pro SM (jede mit 32 FP32 lanes)
- FP32 Operationen pro Lane:
Fused multiply-add (FMA) = 2 flops/Takt
- 2.52 GHz boost clock:
- Theoretisches Maximum: $16384 \text{ cores} \times 2 \text{ flops} \times 2,52 \text{ GHz} \approx 82,6 \text{ TFLOPS}$
- 576 Tensor Cores (Tensor Ops)



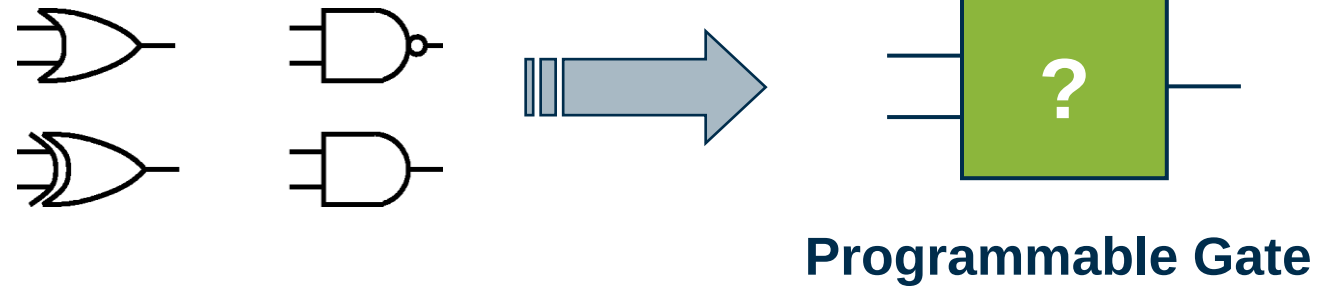
TECHPOWERUP
Source: [Videocardz_AD102](#)

Übersicht – Vorlesung Nr. 03

- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Überblick Technologien
 - Speicherhierarchien
 - Caches

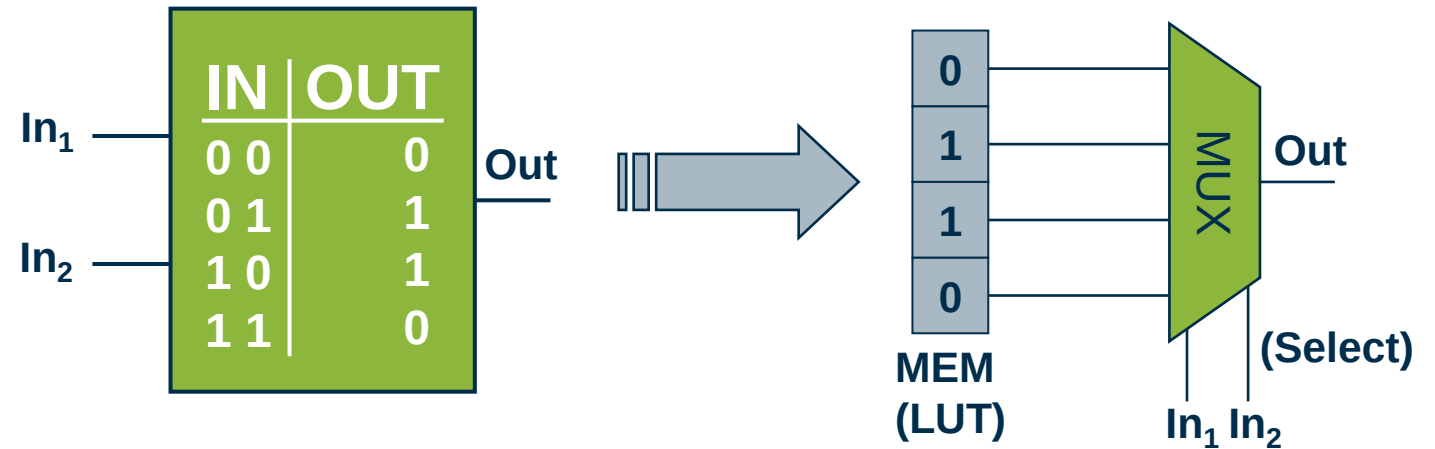
RAM-basierter FPGA : Logikrealisierung

- Menge programmierbarer “Logikgatter”, integriert in ein flexibles



Verbindungsnetzwerk

- “benutzerprogrammierbare” Alternative zu dedizierten Schaltungen
- Look-up-Table (LUT) mit 2 Eingängen



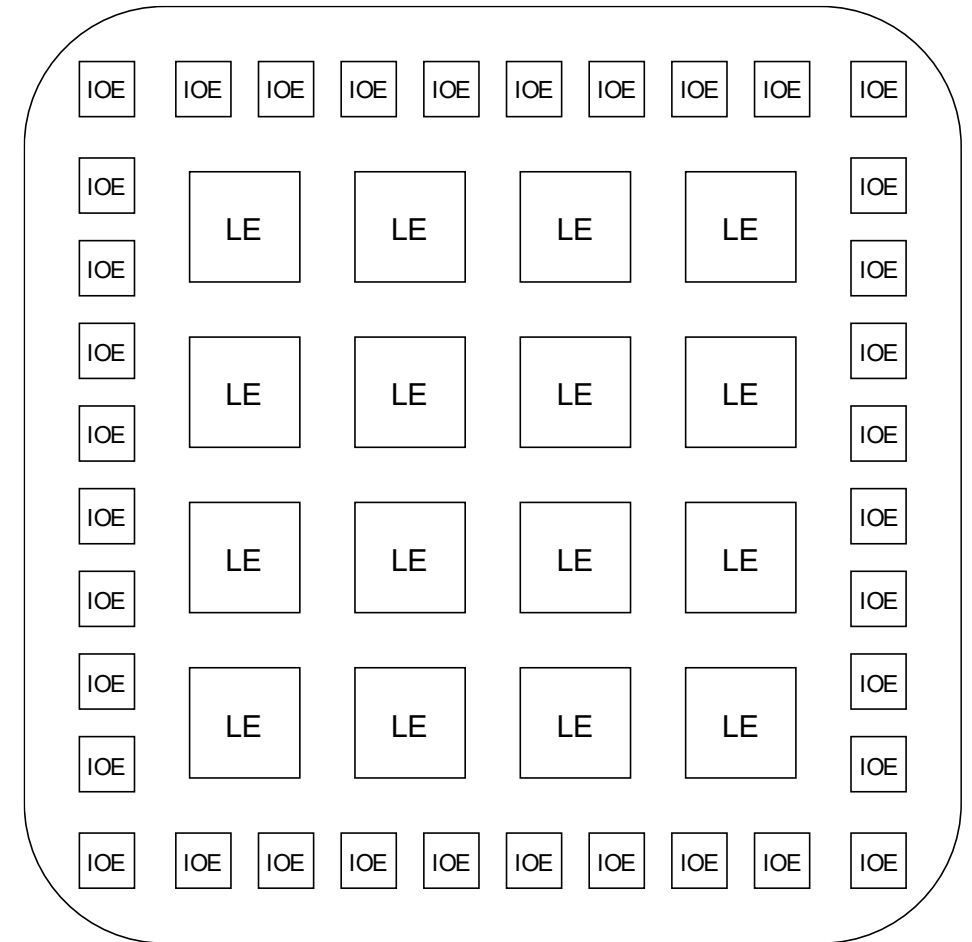
Field Programmable Gate Arrays (FPGAs)

- FPGAs ermöglichen
 - Implementierung von kombinatorischer und sequentieller Logik.
 - Implementierung mehrstufiger Logikfunktionen.
- Moderne FPGAs integrieren zudem eingebaute Multiplikatoren, Hochgeschwindigkeits-I/O-Module, Datenkonverter, RAM Arrays und Prozessoren.

Referenz S. 275 [1]

Field Programmable Gate Arrays (FPGAs)

- FPGAs sind Arrays aus **logic elements (LEs)**, die auch als **configurable logic blocks (CLBs)** bezeichnet werden.
- Jeder CLB kann kombinatorische oder sequentielle Logik ausführen.
- **Input/Output elements (IOEs)** verbinden die Ein- und Ausgänge der CLBs mit den Pins auf dem Chip-Gehäuse.

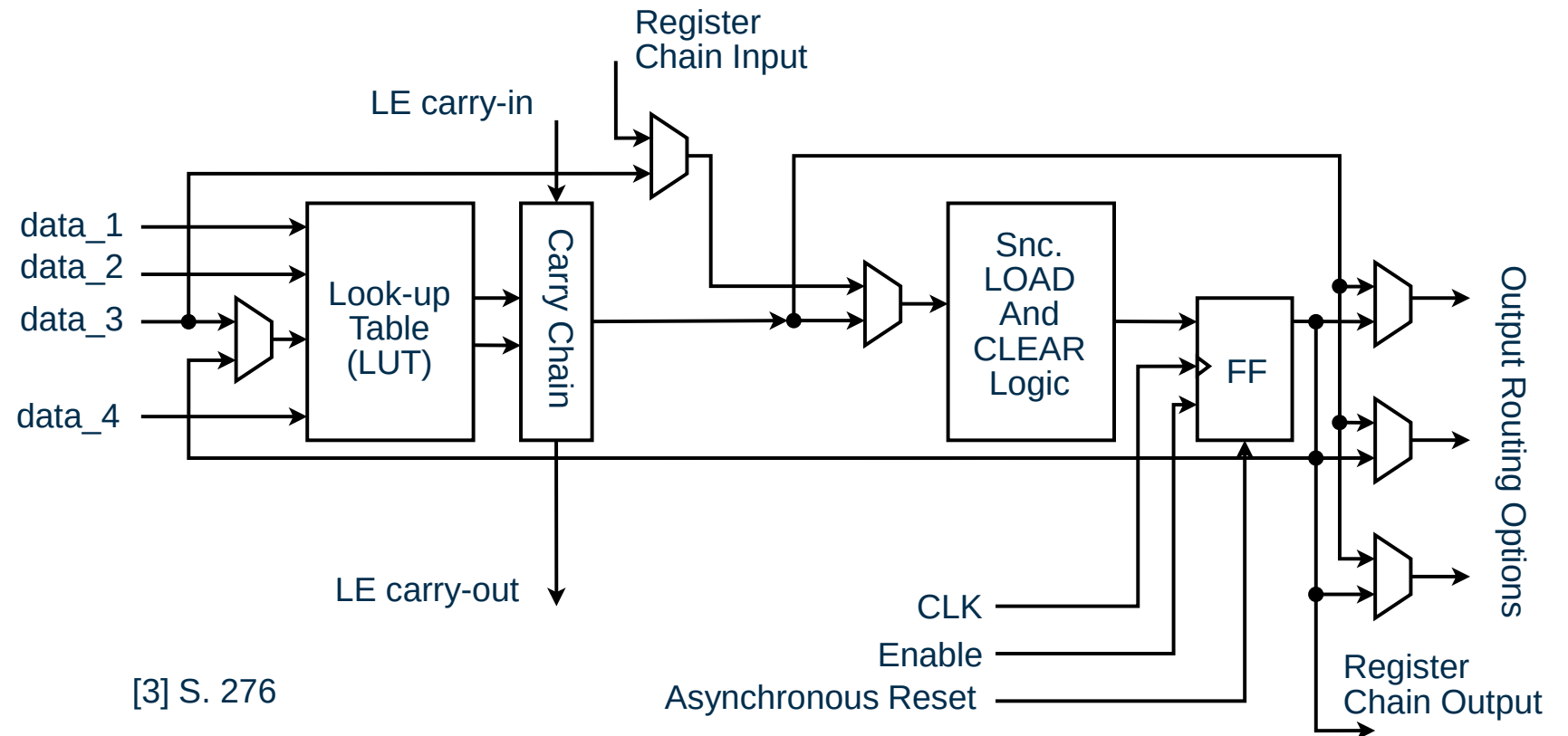


[3] S. 275

Logic Arrays

- Logic elements (LEs) können je nach Kontext unterschiedliche Funktionen ausführen.

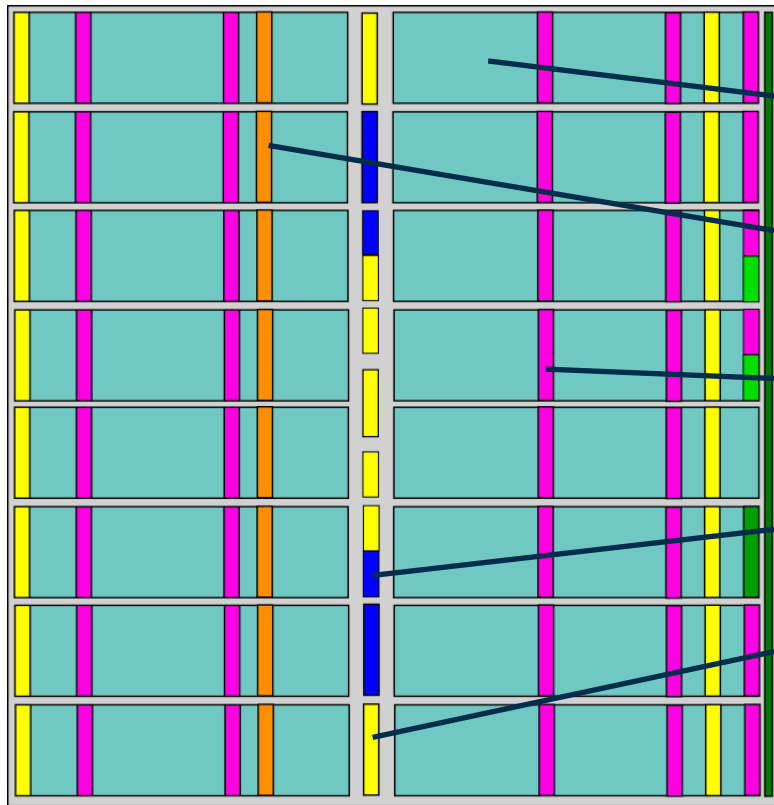
- Grundlegende Logikfunktionen
- Eine Adder Komponente (vorgefertigter Volladdierer)
- Flip-Flop/Register



Target Technology LUT-based FPGA

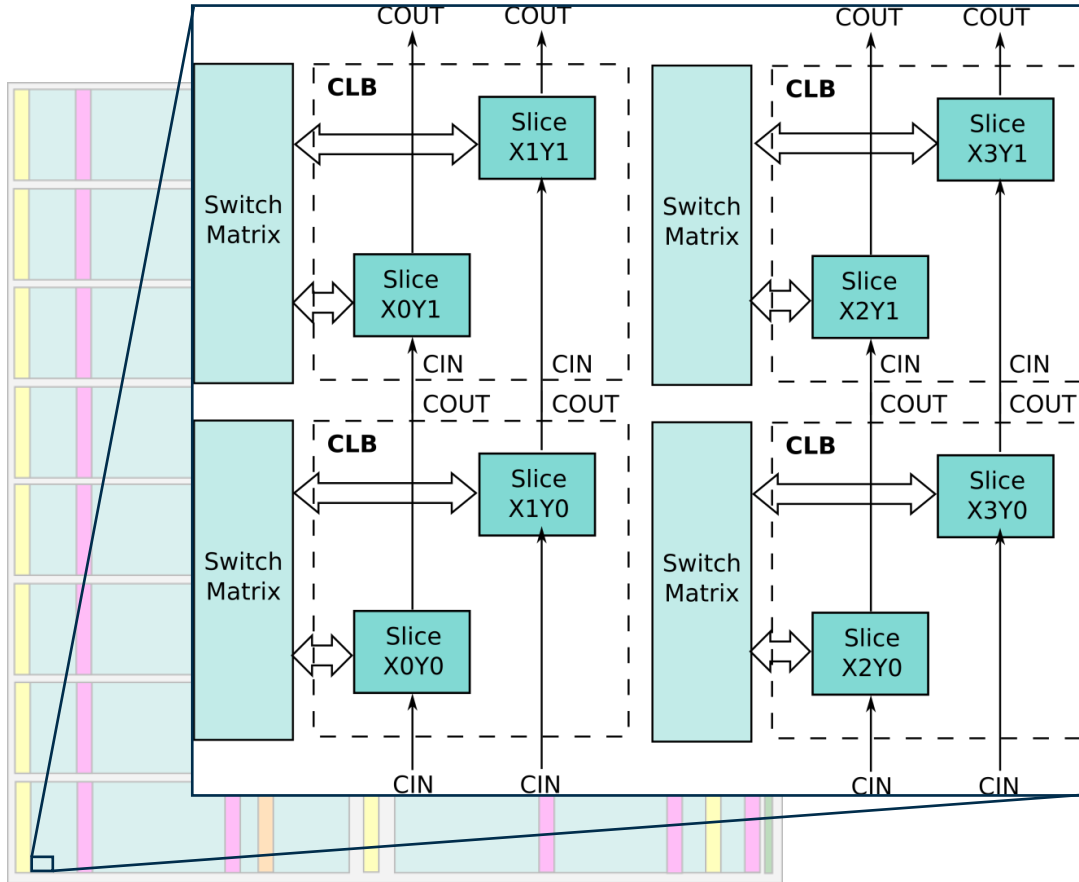
- Die Look-up Table FPGAs implementieren boolesche Funktionen mit LUTs
- Die einzige Einschränkung für die booleschen Funktionen ist die maximale Anzahl von Inputs
 - Diese booleschen Funktionen bilden eine virtuelle Bibliothek
- Für n Inputvariablen gibt es 2^{2^n} verschiedene single-output Boolesche Funktionen
- Beispiel
 - Die Xilinx Ultrascale-Architektur verwendet Lookup-Tabellen mit sechs Inputs und zwei unabhängigen Outputs: $2^{2^6} \approx 18 \times 10^{18}$ mögliche Konfigurationen

FPGA Beispiel: Xilinx Virtex 5



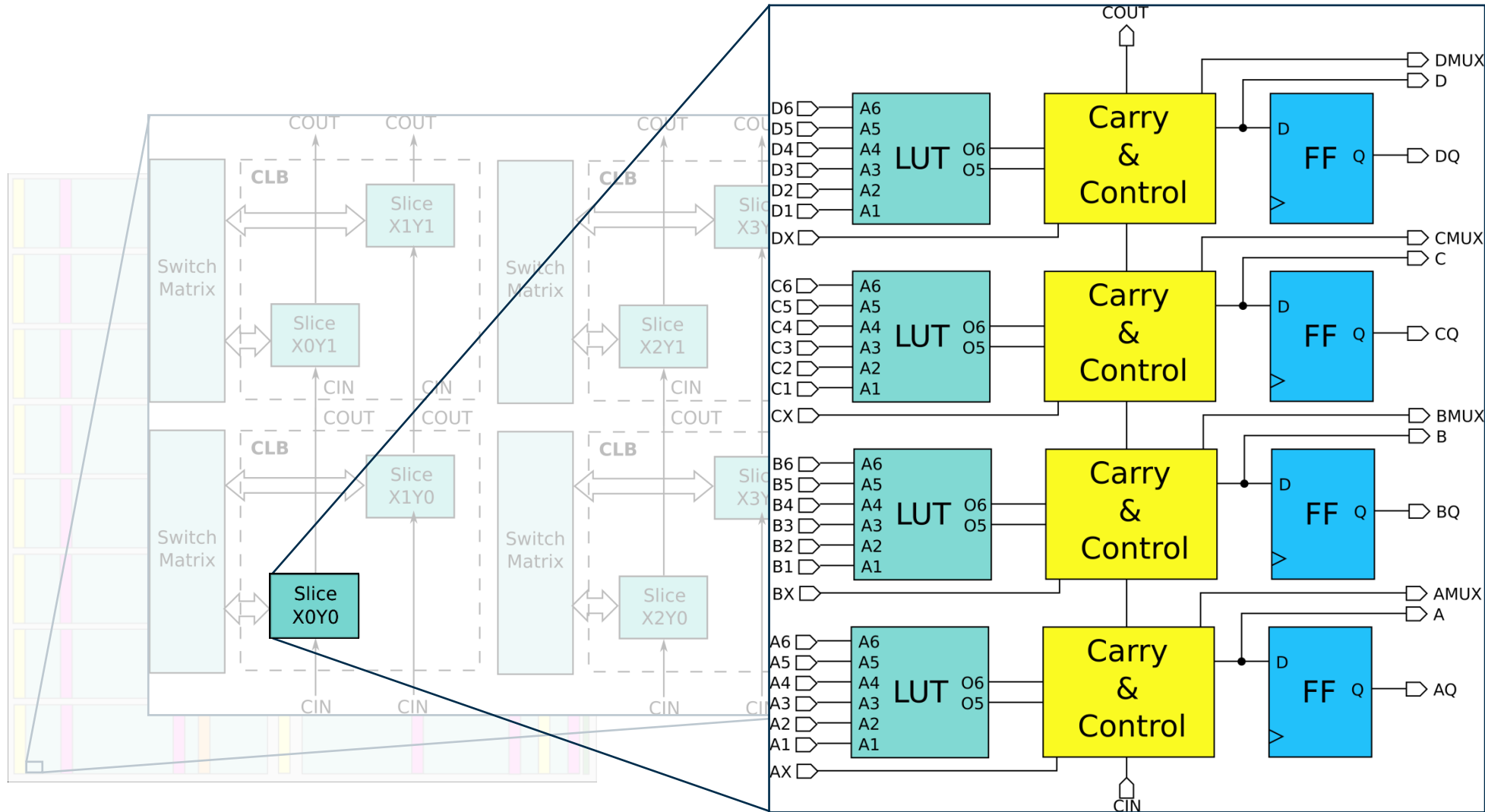
- Configurable Logic Block (CLB)
- Digital Signal Processor (DSP)
- Block RAM (BRAM)
- Digital Clock Manager (DCM)
- I/O Bank

FPGA Beispiel: Xilinx Virtex 5



- Ein CLB besteht aus zwei Slices
- Programmierbare Switch Matrix
- Schnelle Routing innerhalb CLB
- Globales Routing zwischen CLBs

FPGA Beispiel: Xilinx Virtex 5



Übersicht – Vorlesung Nr. 03

- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Speicherhierarchien
 - Überblick Technologien

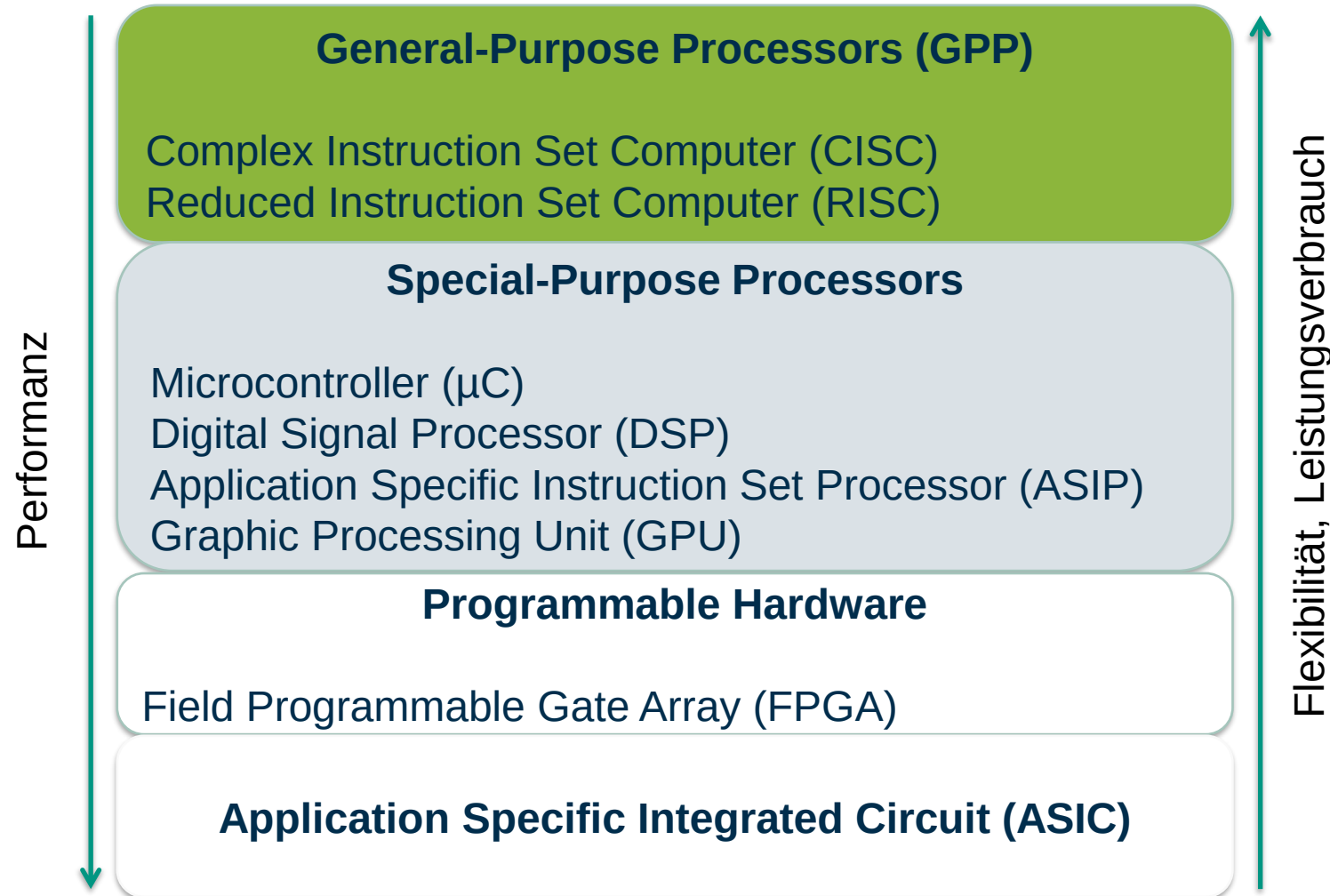
ASICs

- *Application Specific Integrated Circuits* (ASICs) sind *anwendungsspezifische* Chips
- ASICs sind häufig schneller und effizienter als General Purpose Hardware
- Beispiel rechts: Bitcoin Mining

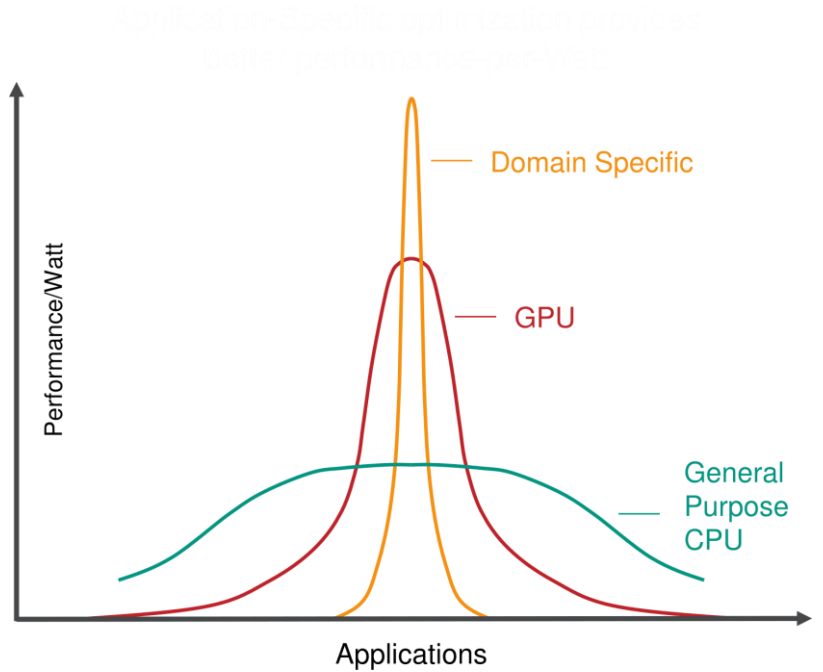
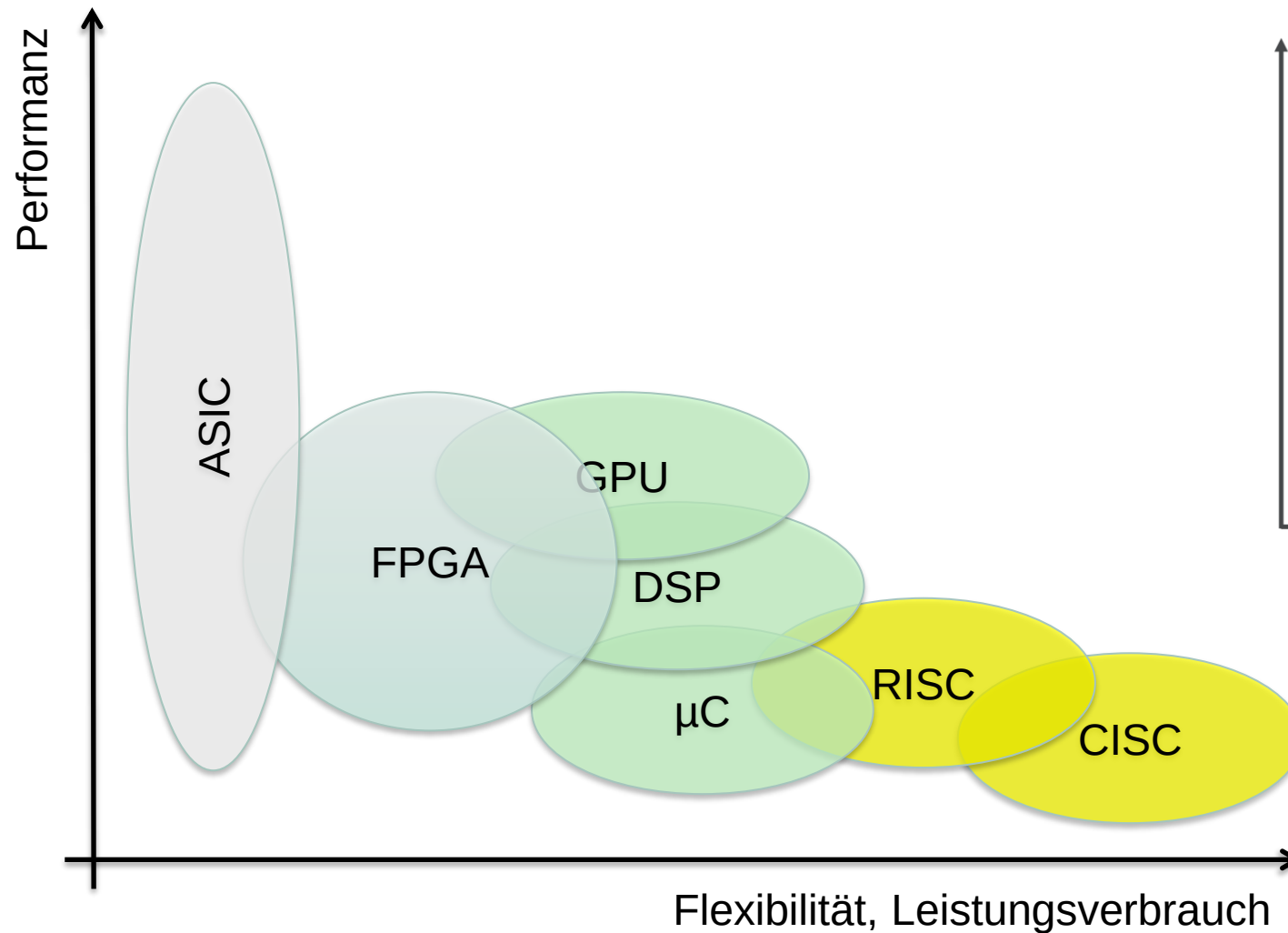
„Ein ASIC-Miner aus 2018, der 2500 \$ kostet, kann 400 GPUs oder 12.000 CPUs ersetzen.“



Vergleich der Zielarchitekturen



Vergleich der Zielarchitekturen



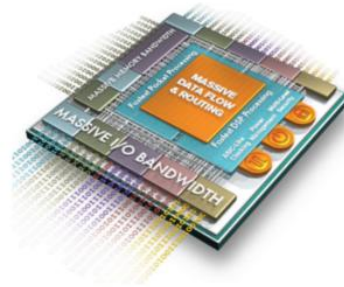
Beispiel AI Hardware Lösungen: Hard- and Soft-IP



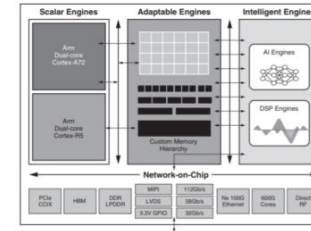
CPU



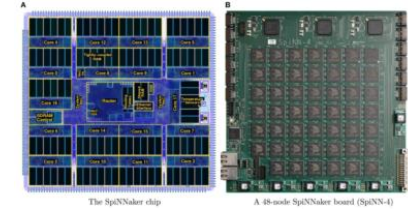
GPU



Soft IP



Hard IP



ASIC

Flexibility/Costs

Efficiency

- Geeignete Plattformen, Architekturen
 - FPGA-Systeme
 - Coprozessor / Heterogenität
- Verfügbarkeit von Komponenten, Technologien und Integration

Übersicht – Vorlesung Nr. 03

- Methoden die Performanz zu erhöhen
 - Mikroarchitekturtechniken: Pipelining, Superskalarität und VLIW
 - Befehlserweiterungen: SIMD
 - Caches
- Spezialisierte Hardware
 - Grafikkarten
 - FPGAs
 - ASICs (In / Near-Memory)
- Speicher
 - Speicherhierarchien
 - Überblick Technologien

Volatile Memory

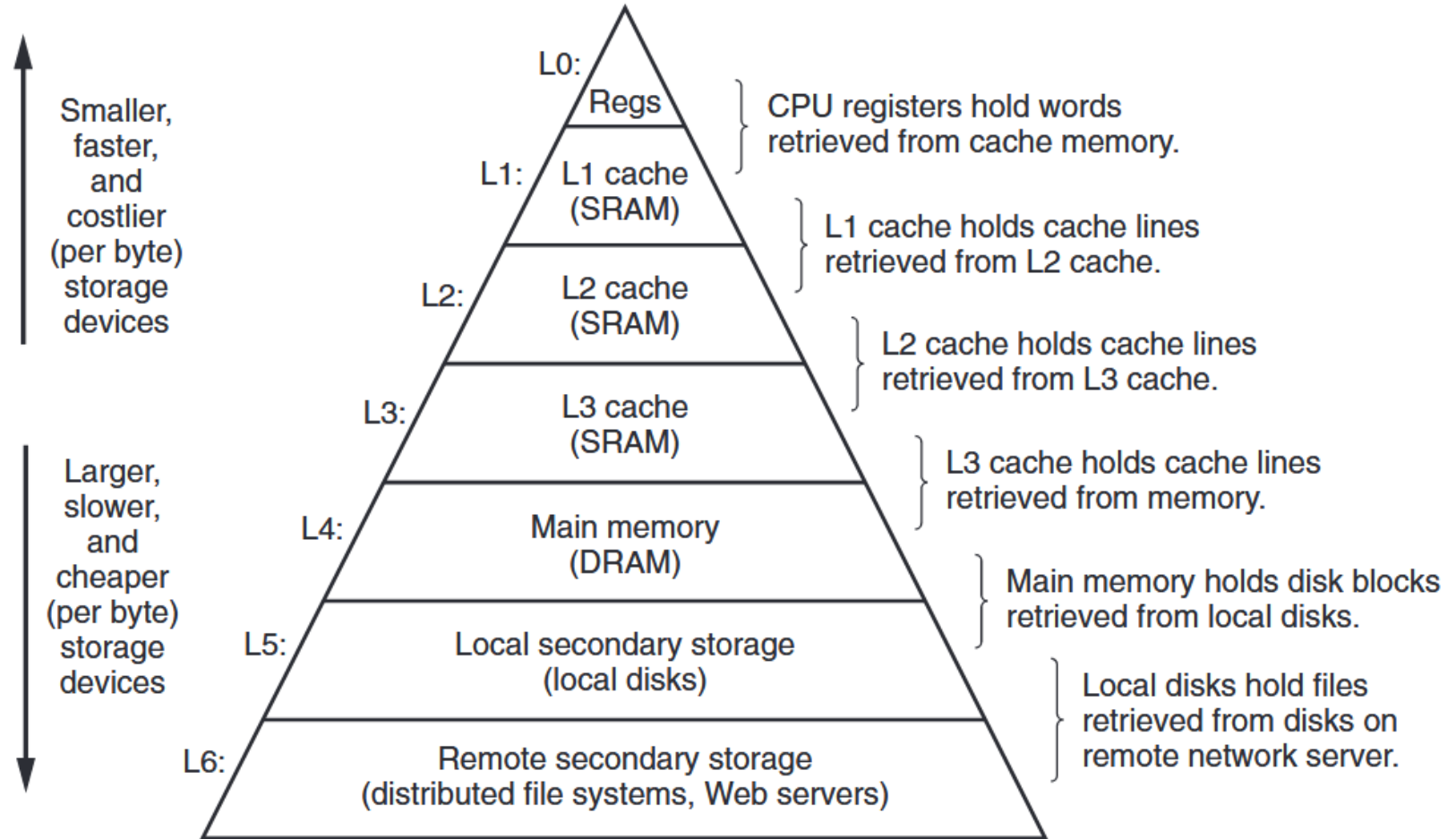
- Die vorgestellten flüchtigen Speicher unterscheiden sich hinsichtlich Flächenverbrauch und Delay Eigenschaften.

Speichertyp	Transistoren pro Bit-Zelle	Latenz
Flip-Flop	~20	schnell
SRAM	6	mittel
DRAM	1	langsam

- Bei Flipflops sind die Daten sofort am Output verfügbar.
- Die Latenz von DRAMs ist länger als die von SRAMs, da die Bitleitung nicht aktiv getrieben wird.

Speicherhierarchie

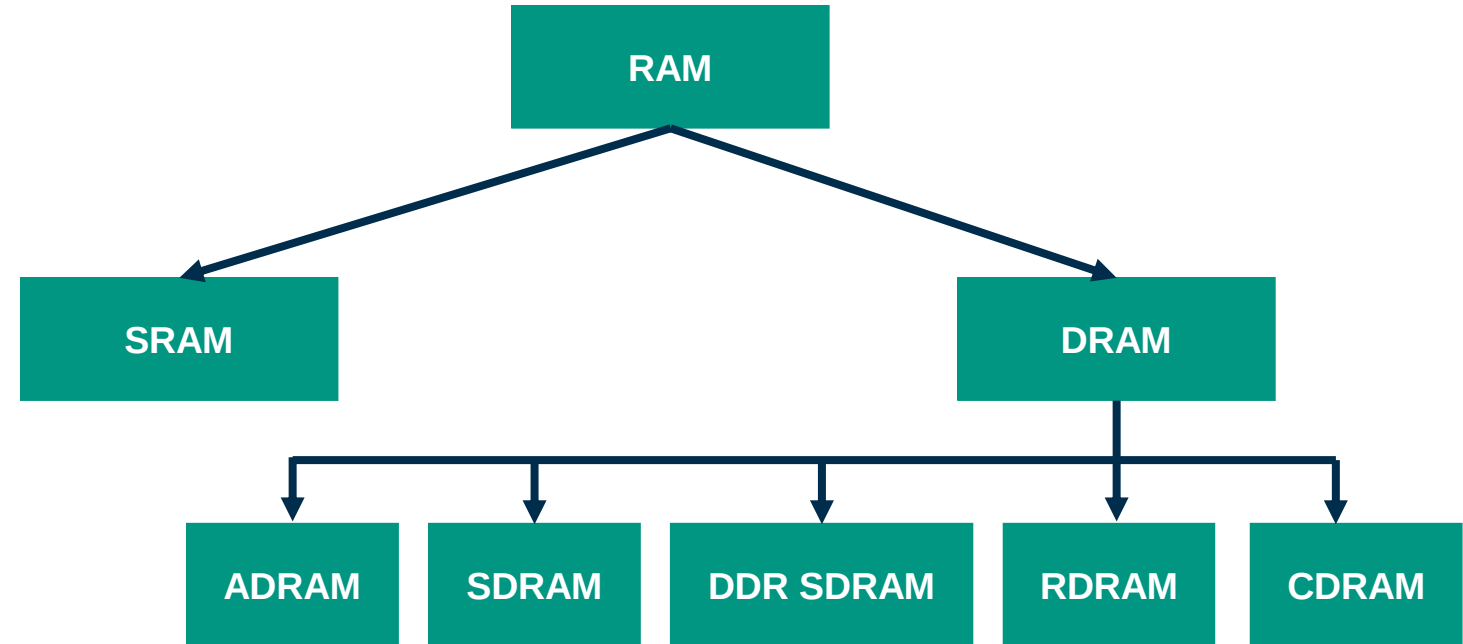
- Speicher ist in eine Hierarchie aufgeteilt
- Der schnellste Speicher ist näher am Prozessor
- RAM ist volatil (kontinuierliche Stromversorgung benötigt)



[1] S.44

Speicher – Technologien

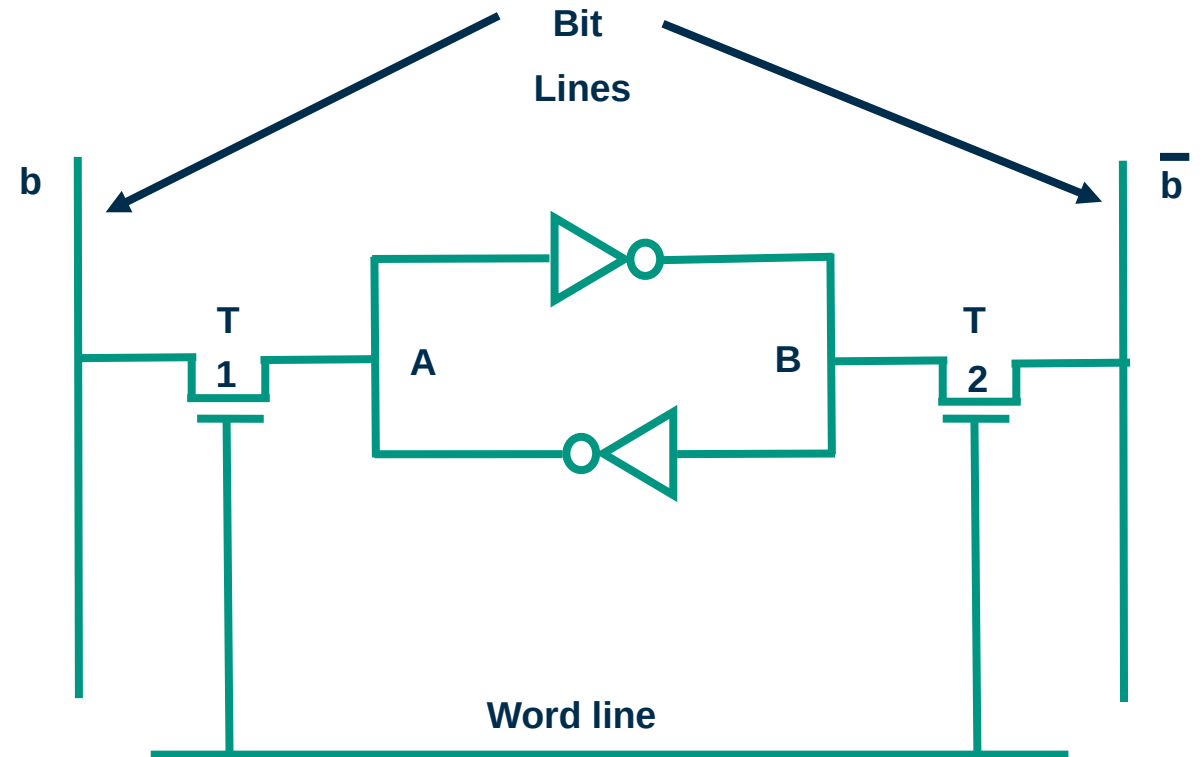
- RAM wird in zwei Hauptkategorien unterteilt: SRAM und DRAM
- SRAM ist sehr schnell und damit gut geeignet für Register und Caches
- DRAM ist günstiger und damit gut geeignet für den Hauptspeicher



[2]

SRAM

- Stabiler Zustand solange Stromzufuhr vorhanden ist
- Erfordert 2 Transistoren und 2 Inverter
 - 6T SRAM cell
- Benötigt viel Fläche
- Die Bit-Zelle ist statisch, da der Bit-Wert aktiv getrieben wird und nicht aktualisiert werden muss



[2]

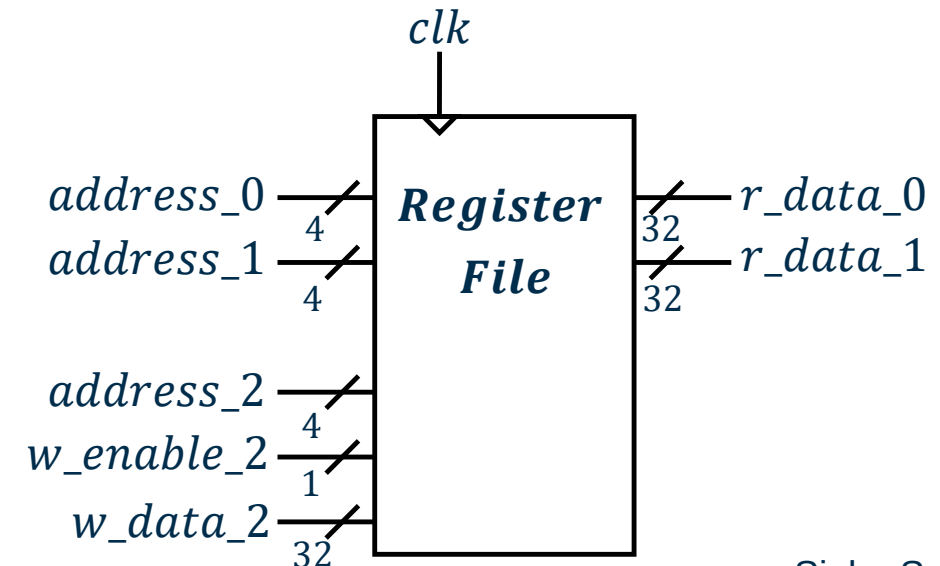
Random Access Memory – Register File

- Eine Gruppe von Registern, sogenannte **Register files**, wird häufig in digitalen Systemen zum Speichern temporärer Variablen verwendet
- Sie wird mit einem **Multi-Port SRAM** Array implementiert, da dies effizienter ist als ein Array aus Flipflops

Beispiel

16x32-Registerdatei mit drei Ports

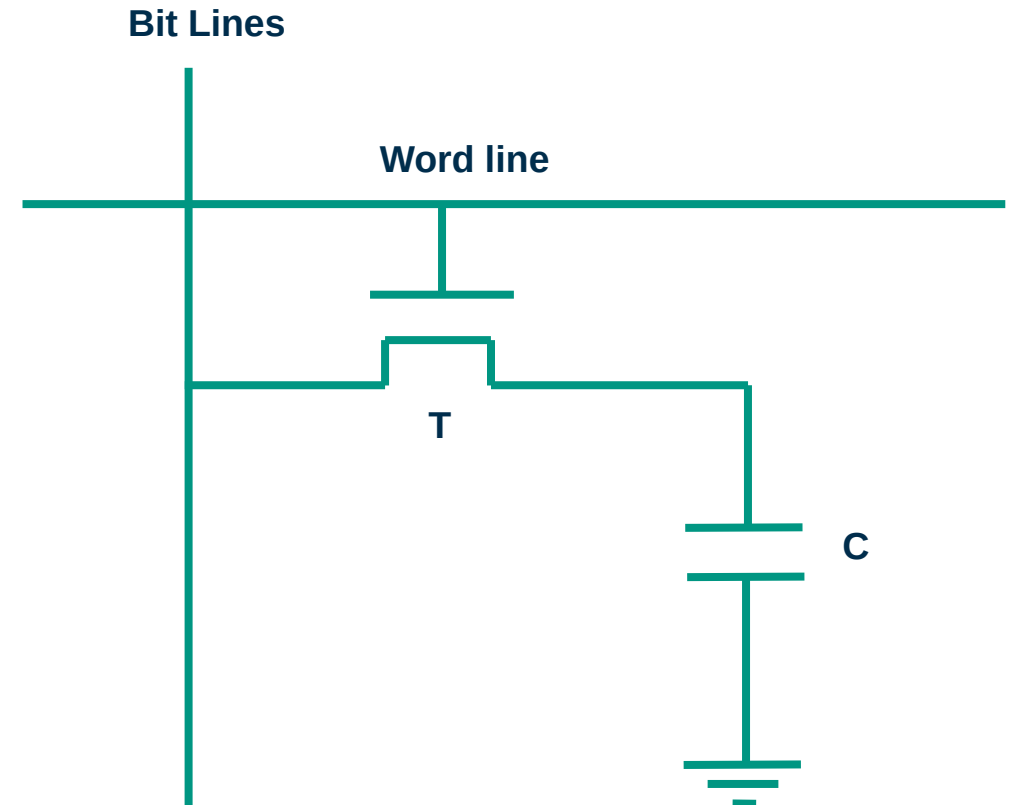
- Es können zwei Register gleichzeitig gelesen und ein Register gleichzeitig beschrieben werden



Siehe S. 268 [1]

DRAM

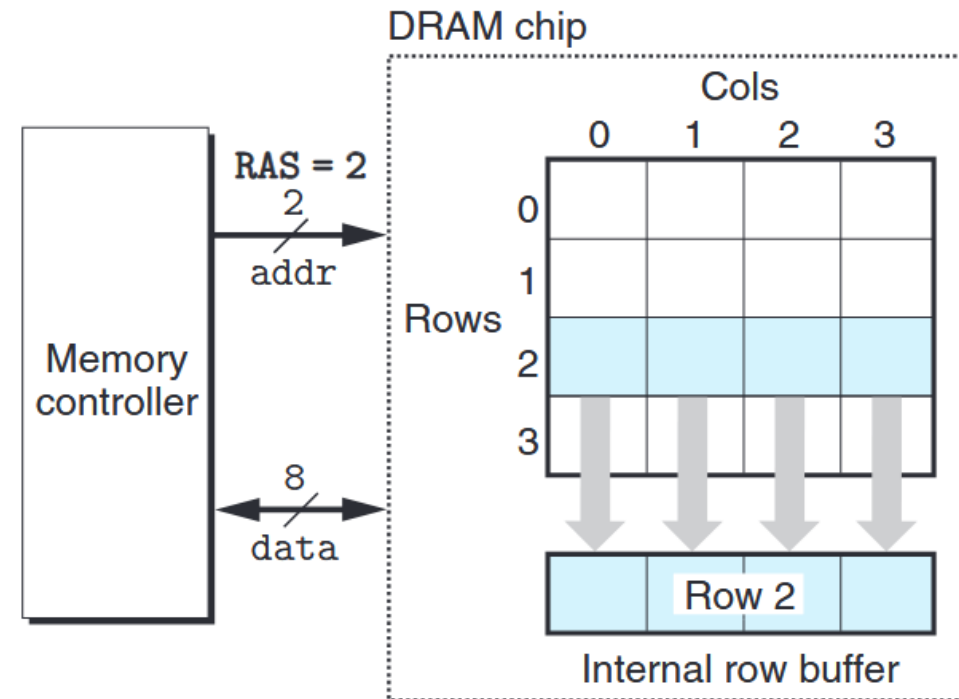
- Speichert Informationen als Ladung von Kondensatoren
- Ladung muss regelmäßig erneuert werden, da sonst Informationen verloren gehen
- Verliert Ladung beim Auslesen
- Verbraucht wenig Fläche (nur ein Transistor und ein Kondensator)
- Die Bit-Zelle ist dynamisch, da der Bit-Wert nicht aktiv von einem Transistor getrieben wird



[2]

DRAM Array laden 1

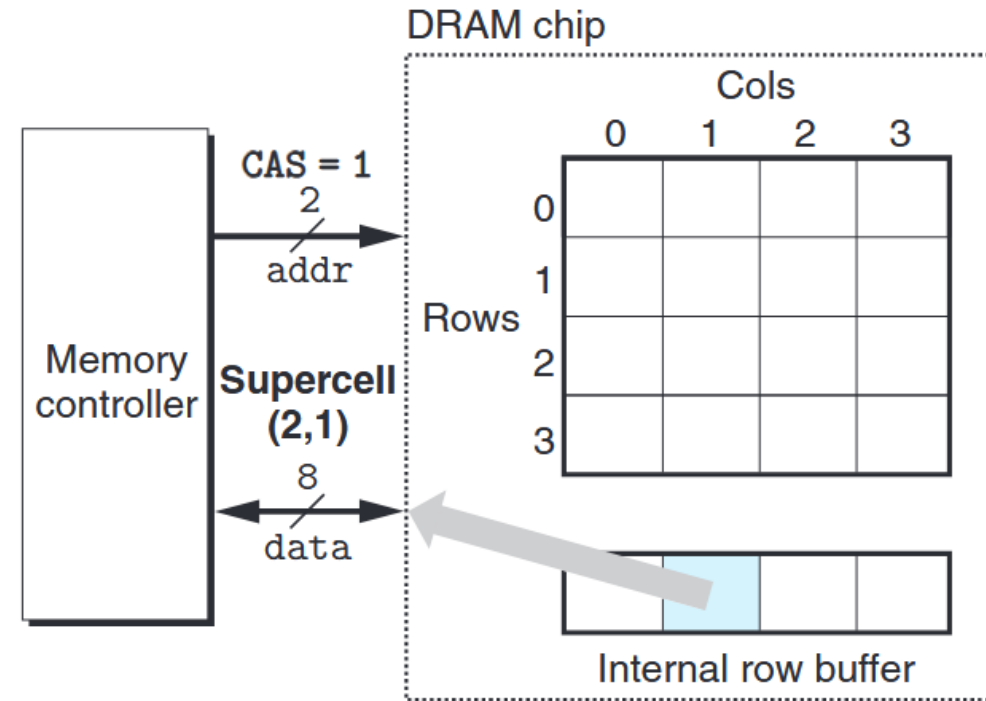
- Zunächst wird die ganze Zeile in einen Buffer geladen
- Dadurch muss der Array lediglich einen Index für die Zeilen haben, nicht für die Spalten



[1] S.612

DRAM Array laden 2

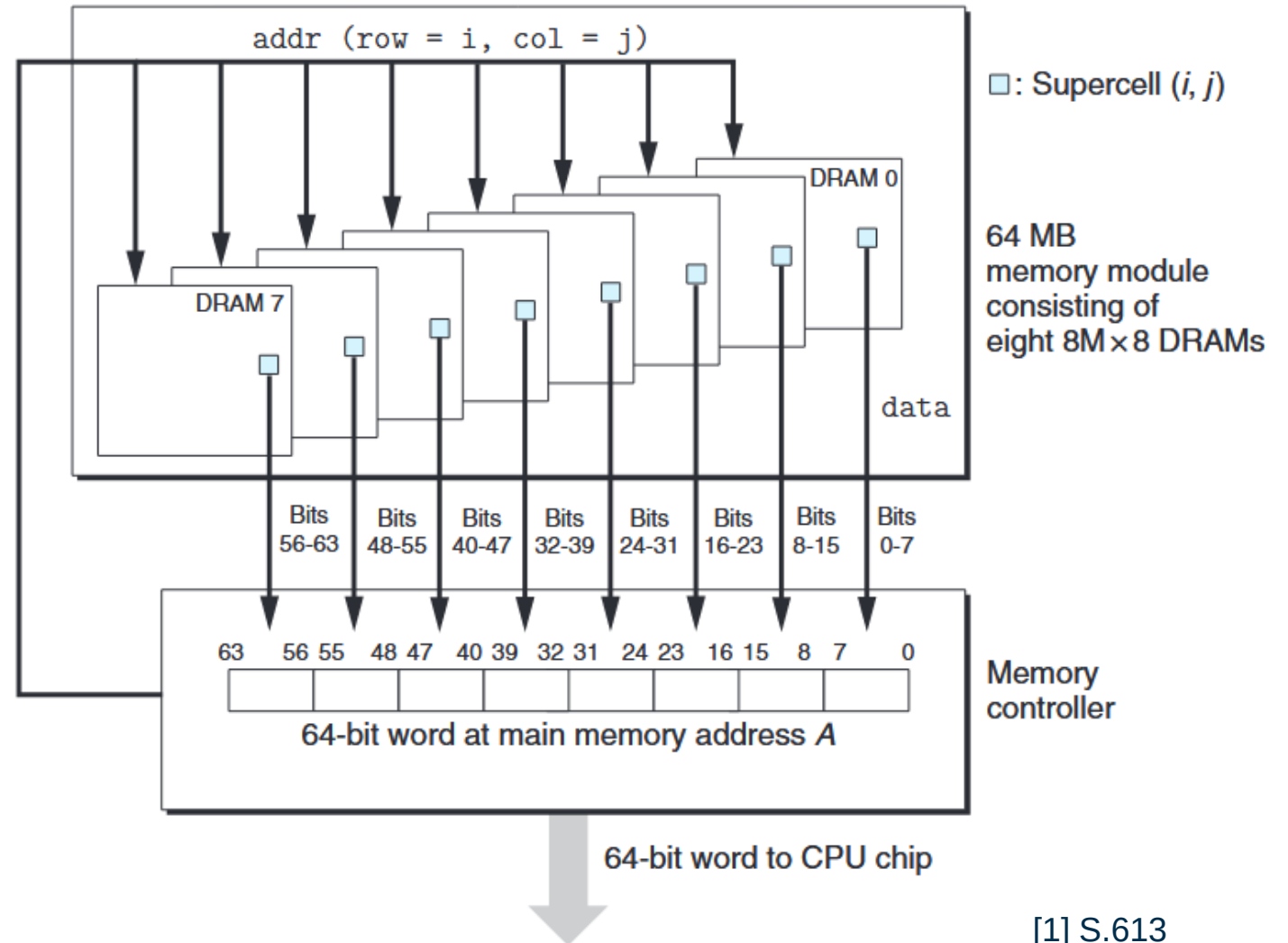
- Anschließend wird die Spalte aus dem Buffer geladen
- Der Buffer muss aus SRAM Zellen bestehen, damit der Wert beim Laden nicht verloren geht



[1] S.612

DRAM Superzellen

- Die Zellen sind tatsächlich Superzellen, welche aus mehreren einzelnen Zellen bestehen
- Das 64-bit Wort aus der Superzelle setzt sich aus den Unterzellen zusammen



Quellen

- [1] Bryant, R. E., David Richard, O. H., & David Richard, O. H. (2003). Computer systems: a programmer's perspective (Vol. 2). Upper Saddle River: Prentice Hall.
- [2] Different Types of RAM (Random Access Memory) <https://www.geeksforgeeks.org/computer-organization-architecture/different-types-ram-random-access-memory/> (Zugriff 12.02.2026)
- [3] S. L. Harris und D. M. Harris, *Digital design and computer architecture*, ARM® edition. Waltham, MA: Morgan Kaufmann, 2016.
- [4] Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 1st edition
- [5] What is an ASIC: A Comprehensive Guide to Understanding Application-Specific Integrated Circuits, Wevolver, <https://www.wevolver.com/article/what-is-an-asic-a-comprehensive-guide-to-understanding-application-specific-integrated-circuits> (Zugriff 23.04.2026)

